2026 年安徽省集成电路精英赛

参赛赛道:	赛道 2-FPGA 设计及应用赛道
作品名称:	基于 Zynq 的数字 IQ 信号实时分析系统
参赛成员:	李喆铭　　岳晗琦
指导老师:	杜高明

目录

1 调研与方案选择 3
1.1 赛题分析 3
1.2 调研内容 3
1.3 开发平台选择 5
1.4 方案选择 5
1.4.1 FFT 计算结构与数据格式 5
1.4.2 时域与频域测量方法 6
1.4.3 数据输入与缓存组织 7
2 系统设计方案 9
2.1 系统架构 9
2.1.1 软硬件分工 9
2.1.2 PS、PL 互连 10
2.2 数据装载 11
2.2.1UDP 分包写入 DDR 与 CRC 累加 11
2.2.2 块提交与 DMA 传输 12
2.2.3PL 接收与完整性校验 13
2.3 IQ 双 Bank 回放 14
2.3.1 窗口边界切换 14
2.3.2 采集结束后的结果排空判断 15
2.4 时钟域、复位、FIFO 与元数据同步 17
2.4.1 复位传递与启动就绪 17
2.4.2 输入 FIFO 与频域结果 FIFO 18
2.4.3 诊断计数与谱快照握手 20
2.4.4 跨域约束与错误反馈 22
2.5 窗函数、定点预处理与 FFT 接口 22
2.5.1 窗乘积缩放与收敛舍入 23
2.5.2 Hann 窗加权与复数定点预处理 23
2.5.3 16384 点 FFT 接口与窗口边界传递 25
2.6 功率谱、索引重排与频点检测 26
2.6.1 功率谱生成 26
2.6.2 频率位置映射 27

2.6.3 频点检测 28
2.7 并行带宽扫描 29
2.7.1 共享功率谱存储与延迟写入调度 29
2.7.2 八路功率的并行前缀累加 31
2.7.3 99% 带宽边界 32
2.8 时域幅度 34
2.8.1 原始 IQ 功率计算与窗口能量累加 34
2.8.2 固定窗口末点处理与统计结果输出 35
2.8.3 峰值、RMS 与定点输出 36
2.9 波形长度检测与完整性标记 38
2.9.1 滑动功率与双门限判断 38
2.9.2 起始确认、活动状态与结束确认 39
2.9.3 数字零检测与完整性标志 40
2.10 结果记录与上位机读取 42
3 功能仿真结果 43
3.1 仿真环境与方法 43
3.2 FFT 输出与频率换算 43
3.3 占用带宽与频谱边界验证 44
3.4 幅度与活动区间长度 46
3.5 接口控制与时延 48
4 实板测试结果 52
4.1 测试平台 52
4.2 资源与时序 53
4.3 测量结果 54
4.3.1 基础指标 54
4.3.2 噪声条件与测试边界 58
4.4 动态切换与稳定性 59
4.5 指标汇总 62
5 创新点 63
6 总结与展望 66
6.1 成果与稳定性 66
6.2 后续优化路线 67

1 调研与方案选择

本作品面向无线通信系统的数字基带 IQ 信号，完成幅度、帧长度、频点和带宽测量。设计时既要保证测量结果有明确的计算依据，也要在 FPGA 上持续处理高速复数样本。因此，方案选择除比较 FFT 的计算速度，还需要同时考虑数据输入、片上存储、测量后处理和结果传输。

1.1 赛题分析

赛题要求 IQ 信号位宽不低于 12 bit，数据速率不低于 100 MSPS，FFT 点数不少于 8192，从接收 IQ 信号到得到频域信息的处理时间不超过 2 ms。在满足基本指标的前提下，进一步提高数据速率、扩大被测信号带宽并缩短分析时间。

本设计将一个样本定义为同一采样时刻的一对 I、Q 数据，采用 I、Q 各 16 bit 的有符号整数格式。按这一格式计算，100 MSPS 对应 400 MB/s 原始数据；将板内处理速率提高到 160 MSPS 后，数据量达到 640 MB/s，即 5.12 Gbit/s。这一计算首先确定了输入方案的限制：千兆以太网不能持续传送同等速率的全新 IQ 数据，网络装载速度与 FPGA 板内处理速度必须分开考虑。

FFT 点数选择 16384。在 160 MSPS 下，每个分析窗口覆盖 102.4 μs，频率栅格间隔为 9.765625 kHz。该配置能够在达到点数要求的同时控制缓存规模和计算延迟。继续增加点数可以细化频率栅格，但会增加观测时间和存储开销；提高采样率则会扩大频谱跨度，同时使固定点数下的栅格变粗。

1.2 调研内容

前期调研主要围绕如下问题展开：8192 点 FFT 如何连续接收样本，定点位宽和缩放如何确定，频谱输出顺序如何处理，四项测量采用什么定义，Zynq 平台如何组织数据，以及怎样验证计算结果。通过查阅论文正文、厂商指南和项目源码，进行调研和对比。

FFT 架构以 Garrido 的 *A Survey on Pipelined FFT Hardware Architectures* 为主要参考。该设计方式将流水结构分为单流和多流两类，便于先按每拍处理的样本数筛选架构，而不是直接比较不同器件上的最高频率。对于本作品的一路复数输入，单样本流水结构与目标吞吐相匹配。

此外还参考了部分开源项目和官方手册。主要资料的研究内容、使用范围及其对方案选择的作用见表 1-1。

表 1-1 调研资料

资料或项目	研究内容	选择依据	使用范围
Garrido：*A Survey on Pipelined FFT Hardware Architectures*	单流与多流流水 FFT 的分类、数据组织及结构取舍	根据单路复数输入和每拍一对 IQ 的目标，选择单样本流水结构	架构分类与比较内容
AMD FFT 产品指南 PG109	流式结构、valid/ready 握手、定点缩放、输出顺序及 XK_INDEX	采用 Pipelined Streaming I/O、固定缩放和位倒序输出；根据 XK_INDEX 定位频点，避免重复倒序	核对 IP 接口与数值语义
Harris：*On the Use of Windows for Harmonic Analysis with the Discrete Fourier Transform*	有限长度观测的谱泄漏，以及窗函数的主瓣与旁瓣特性	保留矩形窗与 Hann 窗，分别用于相干信号检查和泄漏抑制；原始 IQ 幅度统计与加窗分支分开	窗系数、定点舍入及测量结果
MathWorks：obw 官方说明	根据累计功率确定占用带宽，说明频谱估计方式对结果	选择 99% 等尾占用带宽；硬件参考模型统一窗函数、	用于确定测量定义

	的影响	分析范围和离散边界规则	
nanamake/r22sdf、owocomm-0/fpga-fft、ZipCPU/dblclockfft	R2²SDF、四步 FFT、参数化位宽与使能控制等实际实现	比较自主 FFT 的存储映射、乘法流水、接口和工具适配成本	检查固定版本源码及相关文件
Digilent Zybo Z7 官方参考手册	Z7-20 器件、DDR3L、以太网、启动方式及板级接口	选择 Zybo Z7-20，采用 PS 管理数据和控制、PL 完成实时测量的分工	用于确认平台条件

1.3 开发平台选择

平台选择考虑两类路线：纯 FPGA 完成运算和外围控制，或采用 Zynq 将实时运算与控制软件分开。由于 Zynq 的 PS 可承担控制任务，PL 则保留给逐样本运算和确定性时序，较适合本作品需要反复更换测试信号、读取结果并进行自动化验收的开发方式，因此采用 Zynq 作为开发平台。板卡层面，前期比较了 ALINX AX7020、AX7A200B 等，最后结合已确定的硬件条件选择 Zybo Z7-20。AX7A200B 的逻辑、DSP 和片上存储资源更多，适合增加并行度或缓存，但不能直接沿用 Zynq PS 软件；而 Zybo Z7-20 同时具备 XC7Z020、DDR3L、千兆以太网和 microSD 启动条件，可在一块板上完成数据装载、硬件分析及结果回传。

最终采用 Zybo Z7-20，器件为 XC7Z020CLG400-1。PS 负责网络协议、DDR 缓冲、DMA 控制和结果读取，PL 完成幅度、长度、FFT 及频谱参数计算，上位机负责测试输入与显示。

1.4 方案选择

1.4.1 FFT 计算结构与数据格式

FFT 候选方案主要包括存储复用、单样本流水和多样本并行三

类。存储复用结构可以减少运算单元，但需要额外核算连续输入下的处理周期与缓存调度；多样本并行结构能够提高吞吐，同时增加运算和换向复杂度。本作品以 160 MHz 下每拍处理一对 IQ 为目标，选择单样本流水结构。

通过前期调研可知，自主路线仍需完成大点数延迟线映射、旋转乘法流水、接口暂停和定点验证，因此当前工程采用 AMD FFT IP，配置为 16384 点 Pipelined Streaming I/O，使用定点运算、固定缩放和收敛舍入，并输出频点索引。该结构允许不同帧的输入、计算和输出重叠。

为确定位宽与输出顺序，前期在同一 XC7Z020 器件上完成三种独立 FFT 配置的布线后比较，结果见表 1-2。

表 1-2 配置比较

配置	LUT	BRAM36 等效块	DSP
24 bit，位倒序输出	5259	16	36
24 bit，自然序输出	5432	27	36
18 bit，位倒序输出	4265	14	36

表 1-2 中，24 bit 位倒序配置比自然序节省 11 块等效 BRAM；相比 18 bit 配置，仅增加 2 块等效 BRAM，DSP 数量不变。因此，最终采用 24 bit 数据通路、18 bit 旋转因子、固定缩放和收敛舍入，在存储开销与数值精度之间取得平衡。

1.4.2 时域与频域测量方法

幅度直接由原始 IQ 计算。单样本功率为 I^2+Q^2，通过峰值搜索和区间能量累计得到峰值幅度与 RMS。相比从频谱反推幅度，该方法不需要重新引入窗函数增益和 FFT 缩放，计算对象更明确。时域分支始终使用未加窗 IQ，输出单位为数字码值。

帧长度定义为波形活动区间长度，而不是通信协议帧的字节数。

前导相关和协议解析依赖已知帧结构，本作品采用适用范围更直接的两种检测模式：能量门限模式通过滑动能量、启停迟滞和连续确认减少边界抖动；数字零模式用于背景严格为零的测试信号，以最后非零样本确定正常结束位置，两者分别对应不同输入条件。

频点取所选分析频率范围内的最大功率 bin；当分析范围覆盖全频带时，对应完整 FFT 功率谱的最高谱峰，同时给出占用频带中心。谱峰搜索便于流水实现，也便于通过单音和双音验证。

带宽采用 99% 等尾占用带宽。相比依赖峰值门限的 3 dB 带宽，累计功率方法更适合描述多峰或起伏较大的通信频谱。硬件按频率累计功率，确定约 0.5% 和 99.5% 的累计功率边界，再计算带宽。

频域分支保留矩形窗与 Hann 窗。矩形窗用于相干单音等检查，Hann 窗用于减轻非相干信号的旁瓣泄漏，同时接受主瓣展宽。窗函数会改变测得的频谱分布，因此硬件与参考模型使用相同系数及舍入方式。针对 16384 点窗口，当前实现利用 Hann 系数的对称和互补关系，仅存储四分之一周期的 4096 个系数，并采用分 bank 的分布式 ROM 组织，以控制系数存储开销并改善读取路径时序。

1.4.3 数据输入与缓存组织

最终装载路径为：上位机 UDP → PS DDR → AXI DMA MM2S → PL 非活动 IQ Bank。活动 Bank 为分析链路供数，非活动 Bank 接收下一块数据；在循环回放模式下，新块未准备好时继续回放旧块。两个 Bank 各最多保存 32768 对 IQ，当前合法装载长度为 16384 或 32768 对 IQ，分别对应一个或两个 FFT 窗口。新块仅在 16384 样本窗口边界激活，避免同一 FFT 窗口混入新旧数据。

软件检查分块 CRC、offset 顺序和整块提交条件，硬件检查装载字数、有效字节、末字标志及整块 CRC。接收结束后，通过独立的分阶段提交逻辑捕获检查信息、判断合法性并生成提交结果，再由统一的 Bank 元数据模块更新长度、块编号、CRC 和就绪状态。

为避免配置两份完整功率谱所带来的存储开销，频谱后处理采用单份完整功率谱存储与延迟写入队列。上一窗口进行带宽扫描和频谱快照读取期间，需要延后写入的新窗口数据暂存在分布式存储队列中；保护解除后，再将队列数据回写到共享功率谱存储。

带宽扫描采用八路并行读取，将 16384 个频点划分为 2048 组，通过平衡前缀累加保留每个频点的累计功率值，使带宽边界仍能定位到单个频点。频谱显示另外将每 16 个频点聚合为一个快照点，形成 1024 点显示数据，带宽计算不依赖降采样后的显示曲线。

2 系统设计方案

2.1 系统架构

系统由上位机、处理系统 PS 和可编程逻辑 PL 三部分组成。上位机负责准备测试数据、设置分析参数和显示结果；PS 负责以太网通信、DDR 数据管理及硬件控制；PL 负责样本回放、时域统计、FFT 变换和频谱后处理。IQ 回放数据分为两条支路：时域支路直接统计原始 IQ 的幅度和活动区间，频域支路通过输入 FIFO、窗函数、FFT 和功率谱扫描提取频谱参数。网络接收与板内计算之间设置存储缓冲，使数据上传和信号分析能够按各自的工作节拍运行。

2.1.1 软硬件分工

上位机通过以太网向开发板发送 IQ 数据和控制命令。PS 接收数据后，将其写入 DDR 缓冲区，并完成数据包校验、块长度检查和上传顺序检查。完整数据块提交后，PS 启动 AXI DMA，将 DDR 中的数据搬运至 PL 内部的 IQ 存储区。PL 从存储区连续读出样本，送入分析电路。

采用这一组织方式，主要考虑到网络数据以数据包为单位到达，包间存在时间间隔，而 FPGA 分析电路需要按固定时钟节拍接收样本。DDR 用于暂存网络上传的数据，PL 内部 IQ 存储区用于提供稳定的回放数据。两级存储将网络传输过程与板内样本处理过程分开，网络到达时间的变化不会直接改变正在回放的数据块内各样本的输出间隔。

PL 回放电路工作在 160 MHz 源时钟域，正常运行时每个时钟周期发出一对 IQ 样本，对应板内回放速率为 160 MSPS。输入 FIFO 用于衔接回放和 FFT 接收节拍；若发生无法接收的情况，系统记录拒绝和完整性错误，而不是将其视为正常降速。

PS 与 PL 的任务划分以数据处理特点为依据。网络协议、参数设置、状态查询和文件交互由软件完成，便于管理和修改；逐样本平方、

能量累加、FFT 及功率谱扫描由硬件完成，以保持连续的数据处理节拍。

2.1.2 PS、PL 互连

PS 与 PL 之间设置寄存器控制通路和 IQ 数据搬运通路，分别承担参数配置、状态读取和样本传输任务。AXI DMA 位于 PL 中，由 PS 配置和启动，负责将 DDR 中的 IQ 数据搬运至分析器内部存储区。

寄存器控制通路由 PS 的 M_AXI_GP0 接口发起，通过 AXI 互连访问分析器外设和 AXI DMA 的控制寄存器。PS 利用该通路设置回放长度、窗函数、分析频率范围及活动区间检测参数，并执行启动、停止、Bank 切换请求和状态读取。分析器外设的基地址为 0x40000000，地址空间为 256 KiB；AXI DMA 控制寄存器的基地址为 0x40400000。分析结果缓冲区映射在分析器地址空间内，由 PS 通过 GP0 接口读取。

IQ 数据搬运采用 AXI DMA 的 MM2S 通道。DMA 通过自身的 M_AXI_MM2S 主接口，经 PS 的 S_AXI_HP0 高性能接口读取 DDR，将内存中的 IQ 数据转换为 32 bit AXI-Stream 数据流，送入分析器的 S_AXIS_IQ 接口。每个数据字对应一组 IQ 样本，低 16 bit 为 I 分量，高 16 bit 为 Q 分量。

DMA 输出进入 PL 装载器，接收到的数据先写入目标 IQ Bank，同时进行长度、字节有效标志、结束标志和 CRC 检查。数据写入存储并不意味着已经允许回放，只有终端检查和提交完成后，目标 Bank 才被标记为有效。活动 Bank 的回放端口与非活动 Bank 的装载端口独立工作，使后台更新与前台分析能够重叠进行。

当前板级设计保留 PS 的 125 MHz 时钟配置，由 FCLK_CLK1 驱动 Clocking Wizard/MMCM，分别生成源与总线域、FFT 与频谱域使用的两路 160 MHz 时钟。源与总线域承担回放、时域测量、结果构建及相关 AXI 逻辑，FFT 域承担窗函数、FFT 和功率谱扫描。

MMCM 的 locked 信号接入复位控制器，时钟稳定后才释放相关逻辑。

2.2 数据装载

数字 IQ 数据从电脑进入 PL 存储区需要经过四个阶段：UDP 分包写入 DDR 并累计 CRC；PS 检查整块提交信息；AXI DMA 将数据送入 PL 装载器；PL 完成终端检查并提交 Bank 元数据。

2.2.1UDP 分包写入 DDR 与 CRC 累加

DDR 数据暂存代码摘录

```
memcpy(
    &dma_buffer[bank][offset],
    &rx[8],
    4 * words
);

upload_crc_state[bank] =
    crc32_update(
        upload_crc_state[bank],
        (const uint8_t *)&rx[8],
        4 * words
    );

upload_next[bank] += words;
```

每组 IQ 样本采用 32 bit 表示，低 16 bit 为有符号 I 分量，高 16 bit 为有符号 Q 分量，数据按照小端格式保存。一个样本占用 4 字节，360 组样本对应 1440 字节有效负载。bank 表示目标 IQ Bank，offset 表示当前分包在完整数据块中的起始位置，words 表示本次分包包含的 IQ 样本数。由于每个 IQ 样本占 4 字节，实际复制长度为 4*words。数据包中的 IQ 负载被复制到 PS DDR 对应的缓冲区，同时使用 crc32_update() 更新该 Bank 的累计 CRC 状态，并增加已经接收的样本数。

工程中每个 type 6 分包最多携带 360 个 IQ 样本，最后一个分包按照实际剩余长度发送。分包接收完成不代表整块数据已经有效，

PS 还需要等待所有分包到齐，再通过块编号、累计样本数和整块 CRC 进行统一确认。

2.2.2 块提交与 DMA 传输

固件检查逻辑代码摘录

```
else if (count != 5 ||
         bytes != 36 ||
         bank > 1 ||
         flags > 1 ||
         length < rd(0x18) ||
         length > 32768 ||
         (length & (rd(0x18) - 1U)))
    error = 1;
else if (upload_block[bank] != block ||
         upload_next[bank] != length ||
         (upload_crc_state[bank] ^ 0xffffffffU) != crc)
    error = 6;
else if (length != rd(0x1c))
    error = 2;
```

固件通过寄存器读取硬件 FFT 点数，检查提交长度不小于一个 FFT 窗口、不超过 32768 对 IQ，且为 FFT 点数的整数倍。当前 FFT 点数为 16384，因此允许的块长度为 16384 或 32768 对 IQ。随后核对块编号、已接收样本数和整块 CRC，并通过寄存器 0x1C 检查提交长度是否与配置的回放长度一致。上述检查不通过时，分别返回对应错误码。

Cache 刷新代码摘录

```
Xil_DCacheFlushRange(
    (UINTPTR)dma_buffer[bank],
    length * 4U
);
```

当 CPU 将数据写入 DDR 后，最新数据可能仍然保存在 ARM 的数据 Cache 中，DDR 中的内容不一定已经更新。如果此时 DMA 直接读取 DDR，可能读到旧数据。因此，在启动 DMA 前，固件执行 Cache 刷新，将 CPU Cache 中的最新数据写回 DDR，确保 DMA

读取到的是刚刚上传的数据。DMA 完成后，固件还需检查 PL 装载器的完成状态、接收数量和计算 CRC；失败时执行异常恢复，不将该块投入回放。

2.2.3PL 接收与完整性校验

PL 装载器锁存每个成功握手的 IQ 数据字，并按低字节到高字节的顺序更新 CRC。CRC 初值为 0xFFFFFFFF，结束时与 0xFFFFFFFF 异或，反射多项式为 0xEDB88320，与 PS 软件对 IQ 负载字节的计算方式一致。当前实现每拍处理锁存字的低字节，再将数据右移一个字节，四次更新完成一个数据字的 CRC 计算。处理期间暂停接收下一数据字，该装载握手节拍与后级 160 MSPS 回放节拍相互独立。

更新 CRC 代码摘录

```
function automatic [31:0] crc32_byte(
    input [31:0] crc,
    input [7:0] data
);
    reg [31:0] c;
    integer bit_index;
    begin
        c = crc ^ data;
        for (bit_index = 0; bit_index < 8; bit_index = bit_index +
1)
            c = (c >> 1) ^ (32'hedb88320 & {32{c[0]}});
        crc32_byte = c;
    end
endfunction
```

装载开始时先使目标 Bank 失效，避免使用尚未完成检查的数据。最后一个数据字完成 CRC 更新后，装载器将最终 CRC、目标 Bank、长度、块编号及错误信息交给独立的 iq_dma_commit 模块。提交模块依次完成信息捕获、合法性检查、目标路由和完成通知；外层控制还检查提交期间的取消和非法操作。在最终提交有效时更新 Bank 的元数据并置位 ready，异常块保持不可回放状态，由 PS 根据状态进行恢复。

阶段	IDLE	CHECK	ROUTE	COMPLETE
当前职责	捕获提交载荷	检查累计错误和最终CRC	生成目标Bank的提交信号	完成状态过渡，返回空闲

每个 IQ Bank 的长度、块编号、CRC 和 ready 由独立的 iq_bank_metadata 实例统一维护。合法提交更新整组元数据；装载失效、取消或 Bank 释放控制 ready 清除；软件写入遵守字节使能和 Bank 可写条件。将元数据更新从 CRC 递推及回放状态机中分离，可以明确寄存器的维护职责，并缩短提交控制路径。

当前实现利用逐字 CRC 处理期间的空隙，提前计算后续数据字的长度范围条件，并对写地址、数据、字节使能和 Bank 写使能同步寄存。

2.3 IQ 双 Bank 回放

经过前级数据接收和 DMA 装载后，完整的 IQ 数据已经进入 PL 内部存储区。本节主要说明：IQ 数据在 PL 中如何存储、当前数据如何按照固定节拍回放和新数据如何在不破坏当前 FFT 窗口的情况下完成切换。

本工程采用两个 IQ Bank，每个 Bank 为 32768×32 bit，最多保存两个 16384 点 FFT 窗口的数据。每个 Bank 使用双端口存储，装载或主机访问使用一个端口，样本回放使用另一个端口。活动 Bank 为分析链路供数，满足可写条件的非活动 Bank 接收后续数据，Bank 激活和释放由采集控制与元数据管理逻辑协调完成。

IQ Bank 的回放读取具有同步读延迟，因此 Bank 选择信号与读请求一同寄存，使切换边界处返回的数据仍对应发出请求时的 Bank。仅对实际提供样本的 Bank 开启回放端口，主机端口也由真实读写请求使能；按字节写入的 RTL 组织用于推断 BRAM 原生端口使能。

2.3.1 窗口边界切换

bank 切换代码摘录

```
if (pending_valid && issued[LOGN-1:0] == N-1) begin
    active_bank <= pending_bank;
    pending_valid <= 0;
    replay_pos <= 0;
    replay_ptr <= 0;

    current_block_id <=
        pending_bank ? bank_block1 : bank_block0;

    switch_count <= switch_count + 1;
    last_switch_tick <= tick;
end
else if (replay_pos == replay_last) begin
    replay_pos <= 0;
    replay_ptr <= replay_start[14:0];
end
else begin
    replay_pos <= replay_pos + 1;
    replay_ptr <= replay_ptr + 1;
end
```

其中，N 为 FFT 点数，LOGN 为对应的索引位宽，当前分别为 16384 和 14。当存在待切换请求且当前到达 16384 点窗口末尾时，电路将新 Bank 设为活动 Bank，从其起始地址重新回放，同时更新数据块编号，并记录切换次数和时间戳。旧 Bank 的 ready 状态由元数据模块根据释放信号清除，不再由回放状态机直接赋值。

没有切换请求时，回放地址按顺序递增，到达数据块末尾后返回配置的起始位置。replay_last 为预先计算并寄存的 replay_length−1，用于判断当前是否到达块末尾，将减一运算与回放比较分开。每个 Bank 仍保存最多 32768 对 IQ，因此 replay_ptr 和 replay_start[14:0] 保持 15 位，不随 FFT 索引位宽一起改变。

2.3.2 采集结束后的结果排空判断

采集停止与结果排空代码摘录

```
// 数量匹配条件单独寄存。
// 当前 N = 16384，LOGN = 14。
```

```
reg drain_counts_match;

always @(posedge clk) begin
    if (acquisition_reset)
        drain_counts_match <= 1'b0;
    else
        drain_counts_match <=
            (completed == (issued >> LOGN)) &&
            ((freq_producer + freq_dropped) == completed);
end

// 以下为采集状态机 case(run_state) 中的 DRAINING 分支。
// 进入该状态时，wait_count 已清零。
DRAINING: begin
    if (wait_count != 65535)
        wait_count <= wait_count + 1;

    if (wait_count >= 1024 &&
        drain_counts_match &&
        !freq_valid &&
        !freq_busy &&
        !burst_busy) begin
        run_state <= STOPPED;
    end
    else if (wait_count == 65535) begin
        host_errors <= host_errors | 32'h80000;
        run_state <= STOPPED;
    end
end
```

issued 表示已发出的 IQ 样本数，当前每个 FFT 窗口包含 16384 个样本，因此通过 issued >> LOGN 得到对应的窗口数，其中 LOGN=14。completed 表示已完成结果构建的窗口数；freq_producer 和 freq_dropped 分别表示频域记录环已完成发布的记录数和未能接收的记录数。只有窗口计算数量与记录处理数量均匹配时，drain_counts_match 才置为有效。

数量匹配条件先经过寄存，再参与采集状态判断，将计数加法与比较逻辑同状态控制路径分开。进入 DRAINING 后不再发出新样本，已进入流水线的数据继续处理；状态机同时检查当前的 freq_valid、freq_busy 和 burst_busy，避免在仍有频域结果到达或记录环尚未完成

写入时结束等待。等待计数达到 1024 且上述条件全部满足后，系统进入 STOPPED。这里的 1024 是最小等待门槛，实际排空时间仍由结果完成情况决定。

2.4 时钟域、复位、FIFO 与元数据同步

IQ 回放、时域测量和 FFT 分析在不同的处理流水中运行。同一组输入样本进入两条支路后，时域统计和频域计算的完成时间并不相同，配置、窗口编号及诊断信息也需要随处理过程保持一致。因此，模块间连接既要传递数据，也要处理时钟域转换、复位先后关系和结果关联。

系统将 PL 分为源与结果域、FFT 与频谱扫描域。两域由同一 MMCM 的不同输出提供 160 MHz 时钟，连续样本和频域结果仍通过异步 FIFO 传递。窗函数与分析范围在采集前完成设置，并在采集期间保持稳定后同步到 FFT 域；诊断数据和快照窗口号采用握手机制传输。

2.4.1 复位传递与启动就绪

复位同步与本地分发代码摘录

```
(* ASYNC_REG="TRUE" *)
reg [1:0] core_reset = 2'b11;

(* MAX_FANOUT=64 *)
reg core_reset_distributed = 1'b1;

(* KEEP="TRUE" *)
reg reset_launch = 1'b1;

always @(posedge src_clk)
    reset_launch <= rst;

always @(posedge fft_clk) begin
    core_reset <= {core_reset[0], reset_launch};
    core_reset_distributed <= core_reset[1];
end

wire frst = core_reset_distributed;
```

```
assign fft_reset = frst;
```

复位先经过源端寄存和两级同步，再由本地寄存器分发，使 FFT 及相关处理模块在自身时钟下完成复位。同步寄存器与分发寄存器分开设置，也便于实现阶段处理复位信号的高扇出连接。该核心复位传递与板级 MMCM 锁定控制共同作用，时钟稳定后释放板级复位，再在各处理域内完成同步和本地分发。

复位撤销后，系统仍需等待 FIFO 内部初始化和 FFT 配置完成。输入 FIFO 的 wr_rst_busy、rd_rst_busy 分别反映写端和读端的复位忙状态，在对应状态结束前禁止读写。FFT 配置完成信号同步回源域后，与输入 FIFO 状态共同形成 core_ready，采集状态机据此离开 WAIT_CORE。

2.4.2 输入 FIFO 与频域结果 FIFO

IQ 输入跨区域缓冲代码摘录

```
wire accepted = valid && !full && !wr_busy;

xpm_fifo_async #(
    .FIFO_MEMORY_TYPE("block"),
    .FIFO_WRITE_DEPTH(4096),
    .WRITE_DATA_WIDTH(32),
    .READ_DATA_WIDTH(32),
    .READ_MODE("fwft"),
    .FIFO_READ_LATENCY(0),
    .CDC_SYNC_STAGES(2),
    .DOUT_RESET_VALUE("0"),
    .USE_ADV_FEATURES("0004"),
    .WR_DATA_COUNT_WIDTH(13)
) input_fifo (
    .rst(rst),
    .wr_clk(src_clk),
    .wr_en(accepted),
    .din(iq),
    .full(full),
    .wr_rst_busy(wr_busy),
    .wr_data_count(input_occupancy),
```

```
    .rd_clk(fft_clk),
    .rd_en(fft_ready && !empty && !rd_busy),
    .dout(fifo_data),
    .empty(empty),
    .rd_rst_busy(rd_busy),

    .sleep(1'b0),
    .injectsbiterr(1'b0),
    .injectdbiterr(1'b0)
);
```

源域在样本有效、FIFO 未满且写端复位结束时写入数据；FFT 域在后级能够接收、FIFO 非空且读端复位结束时读取数据。FIFO 采用 FWFT 模式，非空时队首数据可供后级使用，读使能用于推进到下一个数据字。

写端通过满标志和复位忙标志判断是否接收样本，读端根据 FIFO 状态及后级就绪信号取数。4096 字缓冲能够吸收短期节拍差异，使回放电路与 FFT 输入流水通过队列衔接。

回放源在运行期间按固定节拍发出样本。如果 FIFO 满或处于复位忙状态，当前样本无法写入，电路会增加拒绝计数并记录错误。异步 FIFO 的容量有限，不能消除长期的输入、输出吞吐差异，也不能把拒绝样本视为正常降速。

时域支路直接接收源端有效样本，频域支路则只处理成功进入输入 FIFO 的样本。因此，发生输入拒绝后，两条支路的数据完整性可能受到影响，必须将错误传递到最终结果，不能仅凭窗口编号相同认定测量有效。

频域结果返回 FIFO 代码摘录

```
xpm_fifo_async #(
  .FIFO_MEMORY_TYPE("distributed"),
  .FIFO_WRITE_DEPTH(16),
  .WRITE_DATA_WIDTH(256), .READ_DATA_WIDTH(256),
  .READ_MODE("fwft"), .FIFO_READ_LATENCY(0),
  .CDC_SYNC_STAGES(2), .DOUT_RESET_VALUE("0"),
  .USE_ADV_FEATURES("0000")
```

```
) freq_cdc (
  .rst(frst), .wr_clk(fft_clk),
  .wr_en(raw_freq_valid && !freq_full && !freq_wbusy),
  .din(raw_freq), .full(freq_full),
  .wr_rst_busy(freq_wbusy), .rd_clk(src_clk),
  .rd_en(freq_pop && !freq_empty && !freq_rbusy),
  .dout(freq_data), .empty(freq_empty),
  .rd_rst_busy(freq_rbusy), .sleep(1'b0),
  .injectsbiterr(1'b0), .injectdbiterr(1'b0)
);
```

频谱模块每完成一个窗口，生成一条 256 bit 中间结果，通过深度为 16 的分布式异步 FIFO 返回源域。该结果保存窗口编号、频谱功率和边界索引，不包含整份 FFT 频谱。源域另设深度为 16 的窗口上下文队列和突发队列，分别保存 192 bit 时域窗口统计和 288 bit 活动区间统计，供后续结果构建模块读取。

2.4.3 诊断计数与谱快照握手

诊断数据跨域握手代码摘录

```
xpm_cdc_handshake #(
    .WIDTH(224),
    .DEST_EXT_HSK(0),
    .DEST_SYNC_FF(3),
    .SRC_SYNC_FF(3),
    .INIT_SYNC_FF(1)
) diagnostic_cdc (
    .src_clk(fft_clk),
    .src_in(diagnostic_payload),
    .src_send(diagnostic_send),
    .src_rcv(diagnostic_ack),

    .dest_clk(src_clk),
    .dest_out(fft_diagnostic),
    .dest_req(diagnostic_arrived),
    .dest_ack(1'b0)
);
```

FFT 域锁存的诊断数据整体要传回源域，发送端保持载荷稳定，接收端在握手完成后取得有效数据，从而避免逐个读取变化中的计数器，该机制保证 FFT 域这组快照的一致性。样本数、窗口数和完成

记录数的守恒检查，应在停止并排空后进行，并同时检查拒绝和丢弃计数。

谱快照写入与完成通知代码摘录

```
always @(posedge fft_clk)
  if(snap_we) snapshot_mem[snap_addr] <= snap_data;

xpm_cdc_handshake #(
  .WIDTH(32), .DEST_EXT_HSK(0),
  .DEST_SYNC_FF(3), .SRC_SYNC_FF(3), .INIT_SYNC_FF(1)
) snapshot_cdc (
  .src_clk(fft_clk), .src_in(snap_window),
  .src_send(snap_send), .src_rcv(snap_received),
  .dest_clk(clk), .dest_out(snapshot_window_cdc),
  .dest_req(snap_arrived), .dest_ack(1'b0)
);

if(snap_arrived && snapshot_pending) begin
  snapshot_valid <= 1;
  snapshot_pending <= 0;
  snap_request <= 0;
  snapshot_window <= snapshot_window_cdc;
end
```

频谱快照数据由 FFT 域写入存储器，写入结束后，再通过握手将窗口号传回源域。源域收到完成通知才将 snapshot_valid 置位，使软件能够将曲线和对应窗口关联。

2.4.4 跨域约束与错误反馈

错误锁存与同步代码摘录

```
always @(posedge fft_clk) begin
  if(frst) core_error <= 0;
  else core_error <= core_error | {
    fft_events[2],
    |{fft_events[5],fft_events[3],fft_events[1:0]},
    raw_freq_valid && (freq_full || freq_wbusy),
    spec_fault
  };
end

// 源域时序逻辑中的同步部分
err_sync1 <= core_error;
err_sync2 <= err_sync1;
```

工程对手写同步寄存器添加 ASYNC_REG 属性，并在 cdc.xdc 中对复位、诊断请求、错误状态等指定同步器的首级输入设置局部路径例外。XPM FIFO 和握手组件采用其自身的跨域约束。

异常信息先在产生错误的时钟域内锁存，再同步回源域。这样可以保留短暂出现的错误事件，供采集控制和结果组装使用。输入 FIFO 拒绝、窗口队列满、频域结果队列满、FFT 异常及窗口编号不匹配均纳入错误反馈。

2.5 窗函数、定点预处理与 FFT 接口

输入 IQ 数据经过异步 FIFO 后进入频域处理支路。该支路以 16384 个样本为一个分析窗口，依次完成窗函数加权、定点格式转换和 FFT 运算，输出复数频谱及对应的频点索引，供后续功率谱计算使用。

上述功能由 window_fft 模块及其内部实例化的 AMD FFT IP 完成。window_fft 负责组织输入数据、窗系数和有效标志，FFT IP 负责复数变换。预处理流水与 FFT 输入之间采用有效、就绪握手，当 FFT 暂时无法接收数据时，前级保持当前数据和控制标志，避免样本与窗口边界错位。

2.5.1 窗乘积缩放与收敛舍入

收敛舍入代码摘录

```
function automatic [23:0] round_even(
    input signed [34:0] x
);
    reg signed [34:0] y;
    begin
        y = (x >>> 9)
          + ((x[8:0] > 9'd256)
          || ((x[8:0] == 9'd256) && x[9]));

        round_even = y[23:0];
    end
endfunction
```

窗函数乘法后得到 35 bit 数据，FFT 输入只保留 24 bit，因此需要缩小数据位宽。代码中的 x >>> 9 表示将乘积除以2^9，同时保留负数符号。后面的判断用于完成收敛舍入：被舍弃部分大于半个量化单位时进位；恰好等于一半时，仅当保留结果最低位为 1 时进位，使结果趋向偶数。其数学形式为：$y = \mathrm{round}_{even}\left(\frac{x}{2^9}\right)$。该方法比直接截断更能减小长期量化偏差，使 35 bit 乘积稳定转换为 FFT 所需的 24 bit 定点输入。

2.5.2 Hann 窗加权与复数定点预处理

四分之一周期系数读取与重构代码摘录

```
// N=16384, ROM_BANKS=8, 每个 Bank 深度为 512
(* rom_style="distributed" *)
reg [17:0] hann_rom[0:N/4-1];
wire complement = index[LOGN-1] ^ index[LOGN-2];
wire midpoint = index[LOGN-2] && (quarter_index==0);

if(ce) begin
  i0 <= $signed(iq[15:0]);
  q0 <= $signed(iq[31:16]);
  for(rb=0; rb<ROM_BANKS; rb=rb+1)
    rom_value[rb] <= hann_rom[rb*512+quarter_index[8:0]];
  rom_bank <= quarter_index[LOGN-3:9];
```

```
  complement0 <= complement;
  midpoint0 <= midpoint;
  hann0 <= hann;
  i1 <= i0;
  q1 <= q0;
  coeff <= !hann0 ? 19'd131072 :
           midpoint0 ? 19'd65536 :
           complement0 ? 19'd131072-
                         {1'b0,rom_value[rom_bank]} :
                         {1'b0,rom_value[rom_bank]};
  pi <= i1 * coeff;
  pq <= q1 * coeff;
  din <= {round_even(pq),round_even(pi)};
  if(valid && ready) begin
    index <= index+1;
    quarter_index <= index[LOGN-2] ?
                     quarter_index-1'b1 :
                     quarter_index+1'b1;
  end
end
```

代码只存储 16384 点 Hann 窗的四分之一周期，共 4096 个 18 bit 系数，并按 8 个深度为 512 的分布式 ROM Bank 读取。quarter_index 在相邻四分之一周期中按相反方向推进，complement0 决定是否取互补值，midpoint0 处理四分之一周期边界。矩形窗使用 131072 作为单位增益系数。i0、q0 和 i1、q1 使输入 IQ 与读取、重构后的系数对齐，再经过乘法与舍入形成 FFT 输入。

数学上，处理过程为：$I_w[n] = I[n] \cdot C[n]$，$Q_w[n] = Q[n] \cdot C[n]$。其中$C[n]$是量化后的窗系数。Hann 窗系数近似表示为：$C[n] = w[n] \times 2^{17}$。因此，最终送入 FFT 的数据为：$I_{\mathrm{FFT}}[n] = \mathrm{round}_{even}\left(\frac{I[n]C[n]}{2^9}\right)$，$Q_{\mathrm{FFT}}[n] = \mathrm{round}_{even}\left(\frac{Q[n]C[n]}{2^9}\right)$。

预处理采用四级流水，有效和末点标志与数据共同推进。ce 由末级有效状态和 fft_ready 决定；当 FFT 暂时不能接收数据时，整条预处理流水保持当前值，样本计数只在 valid 与 ready 同时有效时推进。

窗系数采用2^{17}的定点比例，乘积右移 9 位后，FFT 输入整数相对于原始 IQ 约保留 2^8的比例。矩形窗下，FFT 输入等于原始 IQ 左移 8 位；Hann 窗下，按相同比例加权并进行收敛舍入。

2.5.3 16384 点 FFT 接口与窗口边界传递

FFT 变换接口代码摘录

```
fft8192 fft (
  .aclk(clk), .aresetn(!rst),
  .s_axis_config_tdata(
    iq_build_config::FFT_CONFIG_WIDTH'(
      iq_build_config::FFT_CONFIG_WORD)),
  .s_axis_config_tvalid(cfg_valid),
  .s_axis_config_tready(cfg_ready),
  .s_axis_data_tdata(din),
  .s_axis_data_tvalid(v[3]),
  .s_axis_data_tready(fft_ready),
  .s_axis_data_tlast(l[3]),
  .m_axis_data_tdata(fft_data),
  .m_axis_data_tuser(fft_user),
  .m_axis_data_tvalid(fft_valid),
  .m_axis_data_tready(1'b1),
  .m_axis_data_tlast(fft_last),
  .event_fft_overflow(events[2]),
  .event_tlast_unexpected(events[0]),
  .event_tlast_missing(events[1])
);
```

实例化 16384 点 FFT IP，并完成配置、数据输入、数据输出及异常事件信号的连接,将经过窗函数处理的时域 IQ 数据转换为复数频谱。其对应的数学表达式为$X[k] = \sum_{n=0}^{16383} x_w\,[n] e^{-j\frac{2\pi kn}{16384}}$其中，$x_w[n]$为经过 Hann 窗或矩形窗处理的复数 IQ 数据，n 为时域样本位置，k 为频点编号,$X[k]$为对应频点的复数频谱。当前配置字为 16'h5557，总缩放移位为 15。

变换完成后，fft_data输出由 24 位实部和 24 位虚部组成的复数频谱，fft_user 携带对应的频点索引，fft_valid表示当前输出有效，fft_last 标记本帧频谱的最后一次输出。异常事件信号 events 则用于反映运算溢出、输入末点标志缺失或提前等情况，为运行状态监测提

供依据。后续功率谱模块根据输出复数的实部和虚部计算各频点功率：$P[k] = \mathrm{Re}\{X[k]\}^2 + \mathrm{Im}\{X[k]\}^2$结合 fft_user 提供的频点索引，系统进一步完成频率映射、峰值频点搜索和 99% 占用带宽计算。

FFT 输出与功率计算之间的寄存代码摘录

```
always @(posedge fft_clk) begin
  spectrum_data <= fft_data;
  spectrum_user <= fft_user;
  spectrum_last <= fft_last;
  spectrum_valid <= !frst && fft_valid;
end
```

该寄存级将 FFT 输出的数据、频点索引和末点标志同步延后一拍，再交给功率谱模块。spectrum_valid 在复位期间无效，使未初始化的数据不会进入后续计算。数据与控制信号共同寄存，用于分隔 FFT 输出到平方运算输入的连接路径。

2.6 功率谱、索引重排与频点检测

FFT 输出的复数频谱包含各频点的幅度和相位信息。为了获得信号的频域能量分布，系统在 FFT 后设置功率谱处理模块，对复数频谱进行平方求和，并依据频点索引将结果写入对应的存储位置。在此过程中，同步完成选定频率范围内的功率累加和最高谱峰搜索，为后续占用带宽计算提供功率数据、总功率及峰值频点。

本部分主要由spectrum_measure.sv实现。模块接收 FFT 输出数据及其频点索引、有效标志、帧结束标志和溢出信息，采用流水线结构完成连续处理。频点索引到实际频率的换算则在result_builder.sv中完成。

2.6.1 功率谱生成

复数功率计算代码摘录

```
rr <= $signed(data[23:0]) * $signed(data[23:0]);
ii <= $signed(data[47:24]) * $signed(data[47:24]);

power <= rr + ii;
```

将 FFT 输出的复数频谱转换为功率谱，输入数据的低 24 位为实部，高 24 位为虚部，两部分均按有符号数进行平方运算，再将结果相加，对应数学关系：$X[k] = R[k] + jI[k]$，$P[k] = |X[k]|^2 = R[k]^2 + I[k]^2$。

rr 和 ii 分别保存实部与虚部的平方，power 保存二者之和。由于采用时序逻辑中的非阻塞赋值，平方和求和分属两个流水阶段，可以连续处理输入频点。后续谱峰比较和占用带宽计算直接使用这一功率值，无须开平方，减少了硬件运算量。

ROI 筛选与功率对齐代码摘录

```
wire in_roi = q1 >= roi_low && q1 <= roi_high;
roi_power <= in_roi ? power : 48'd0;
q2 <= q1;
v <= {v[1:0],valid};
l <= {l[1:0],last};
ov <= {ov[1:0],user[16]};
```

ROI 外的频点功率置零，筛选结果单独寄存后再进入累计和谱峰比较。q2 与有效、末点、溢出标志随同对齐，避免将频率范围判断和峰值更新串在同一组合路径中。

2.6.2 频率位置映射

频点索引重排代码摘录

```
q0 <= user[LOGN-1:0] ^ LOGN'(N/2);
q1 <= q0;
q2 <= q1;
```

user[13:0] 为 FFT 输出附带的 XK_INDEX，表示当前数据实际对应的频点。对于 16384 点 FFT，原始索引从直流开始，依次排列正频率和负频率；将索引最高位翻转后，可得到从负频率到直流、再到正频率的地址顺序，其数学关系为：$q = (k + 8192) \bmod 16384 = k \oplus 8192$。映射后，原始直流频点 k=0 对应地址 q=8192，负奈奎斯特频点 k=8192 对应地址 q=0。各地址对应的频率为$f(q) = \frac{(q-8192)F_s}{16384}$。

其中，$F_s = 160000000$ Hz。q1 和 q2 用于使索引与功率计算流水线对齐，随后以 q2 确定功率数据的写入位置。这段代码完成的是地址映射，实际排序通过按地址存储、按地址递增读取实现。

2.6.3 频点检测

组合计算代码摘要

```
wire [47:0] wp = roi_power;

wire [63:0] tnext = total + {16'b0, wp};

wire better = wp > peak || (wp == peak && q2 < peakq);
wire [47:0] pnext = better ? wp : peak;
wire [LOGN-1:0] qnext = better ? q2 : peakq;
```

窗口结果锁存代码摘要

```
if(v[2]) begin
  if(bank_ready[write_bank] ||
     (state != WAIT_BANK && read_bank == write_bank))
    fault <= 1;
  if(l[2]) begin
    if(deferred_pending || protect_store) fault <= 1;
    bank_total[write_bank] <= tnext;
    bank_peak[write_bank] <= pnext;
    bank_q[write_bank] <= qnext;
    bank_overflow[write_bank] <= overflow | ov[2];
    bank_frame[write_bank] <= frame;
    bank_ready[write_bank] <= 1;
    write_bank <= !write_bank;
    frame <= frame+1;
    total <= 0;
    peak <= 0;
    peakq <= N-1;
    overflow <= 0;
  end else begin
    total <= tnext;
    peak <= pnext;
    peakq <= qnext;
    overflow <= overflow | ov[2];
```

```
  end
end
```

这段代码在接收有效功率数据时，同步完成窗口总功率累加和最高谱峰搜索。wp 为经过 ROI 筛选的功率值，范围外的频点已被置零，因此统计结果对应所选频率范围。总功率和峰值分别满足：$T = \sum_{q=0}^{16383} P_{\mathrm{ROI}}[q]$，$P_{\max} = \max_{q} P_{\mathrm{ROI}}[q]$。

better 判断当前频点是否需要替换峰值；功率相等时选择索引较小、频率较低的频点，使结果不依赖数据到达顺序。窗口末点到达时，包含当前频点的 tnext、pnext 和 qnext 一并锁存，随后切换元数据槽并清零统计状态。

2.7 并行带宽扫描

完整功率谱采用一份共享存储，由八路 2048×48 bit BRAM 组成。频点索引低三位选择存储通道，高十一位选择通道地址，每次扫描同时读出八个连续频点，16384 点频谱共分为 2048 组。

本部分由 spectrum_measure.sv 与 spectrum_store.sv 实现。八路功率经过前缀累加形成累计功率分布，按 99% 等尾规则确定带宽边界。上一窗口扫描及快照读取期间，共享存储保持受保护状态，新窗口的功率暂存到延迟队列；保护解除后再回写。元数据切换、存储保护和延迟回写共同保证连续处理。

2.7.1 共享功率谱存储与延迟写入调度

共享存储保护与延迟队列代码摘录

```
wire protect_store = bank_ready != 0 || state != WAIT_BANK;
spectrum_deferred_queue #(
  .W(48+LOGN), .BANKS((DEPTH+128+511)/512)
) deferred (
  .clk(clk), .rst(rst), .protect(protect_store),
  .push(v[2]), .din({q2,wp}),
  .pop_valid(deferred_valid), .pop_data(deferred_data),
  .pending(deferred_pending), .overflow(deferred_overflow)
);

// 八路中的一路，lane 取 0 至 7
```

```
spectrum_shared_lane #(.AW(AW)) storage (
  .clk(clk),
  .live_we(v[2] && !rst && !protect_store &&
           q2[2:0] == lane),
  .live_addr(q2[LOGN-1:3]), .live_data(wp),
  .drain_we(deferred_valid && !rst &&
            deferred_data[50:48] == lane),
  .drain_addr(deferred_data[47+LOGN:51]),
  .drain_data(deferred_data[47:0]),
  .re(scan_request),
  .read_addr(request_count[AW-1:0]), .q(a[lane])
);
```

protect_store 表示上一窗口的数据仍需保留。保护期间，新到达的功率与 14 bit 频点地址组合成 62 bit 记录写入延迟队列；当前队列由 5 个深度为 512 的分布式存储 Bank 组成，共 2560 条。保护解除后，deferred_valid 指示队列回写，同时新功率可通过 live_we 直接写入另一端口。完整功率谱仍只有一份，队列仅保存窗口重叠期间需要延后的数据。

共享功率谱通道端口代码摘录

```
always @(posedge clk) begin
  address_a <= re ? read_addr : drain_addr;
  data_a <= drain_data;
  read_a <= re;
  write_a <= drain_we;
  if(write_a) memory[address_a] <= data_a;
  if(read_a) q <= memory[address_a];
end
always @(posedge clk) begin
  address_b <= live_addr;
  data_b <= live_data;
  write_b <= live_we;
  if(write_b) memory[address_b] <= data_b;
end
```

端口 A 在扫描读取与延迟回写之间切换，端口 B 接收保护解除后的实时功率写入。地址、数据和使能分别寄存，扫描读取保持两拍延迟。队列采用局部地址计数和寄存的 Bank 选择控制扇出，并在下一窗口结束前完成排空；溢出、保护期间非法回写或排空未完成均置错误标志。

2.7.2 八路功率的并行前缀累加

并行前缀累加代码摘要

```
if(pipe_valid) begin
  for(j=0; j<8; j=j+1) begin
    if(j % 2 == 0)
      pair_prefix[j] <= {1'b0, power_pipe[j]};
    else
      pair_prefix[j] <= {1'b0, power_pipe[j-1]}
                      + {1'b0, power_pipe[j]};
  end
end

if(pair_valid) begin
  for(j=0; j<8; j=j+1) begin
    if(j % 4 < 2)
      quad_prefix[j] <= {1'b0, pair_prefix[j]};
    else
      quad_prefix[j] <= {1'b0, pair_prefix[(j/4)*4+1]}
                      + {1'b0, pair_prefix[j]};
  end
end

if(quad_valid) begin
  for(j=0; j<8; j=j+1) begin
    if(j < 4)
      prefix[j] <= {1'b0, quad_prefix[j]};
    else
      prefix[j] <= {1'b0, quad_prefix[3]}
                 + {1'b0, quad_prefix[j]};
  end
end
```

8 个相邻频点的功率组织成三级前缀累加结构,第一级得到每 2 个频点的功率和，第二级扩展为每 4 个频点的累计值，第三级形成 8 个频点范围内的累计功率。其最终结果可以表示为：$C_i = \sum_{m=0}^{i} P_m, \quad i = 0,1,\ldots,7$,其中$P_m$为当前八路输入中的第 m 个频点功率，$C_i$为从该组第一个频点开始的累计功率。

代码将每一级结果寄存，并通过 pipe_valid、pair_valid 和 quad_valid 对齐有效信号。组内前缀与此前各组累计值相加，得到各

频点的完整累计功率，组间反馈只保留一级 64 bit 加法。相同流水同时计算八点最大功率，为快照聚合提供数据。

累计功率反馈与快照聚合代码摘录

```
if(prefix_valid) begin
  cdf <= cdf + {13'b0,prefix[7]};
  for(j=0; j<8; j=j+1)
    cdf_bin[j] <= cdf + {13'b0,prefix[j]};
  compare_addr <= prefix_addr;
  if(capturing) begin
    snapshot_max <= snapshot_next;
    if((prefix_addr & (GROUPS-1)) == GROUPS-1) begin
      snap_we <= 1;
      snap_addr <= prefix_addr >> GROUP_SHIFT;
      snap_data <= {16'b0,snapshot_next};
    end
  end
end
```

当前 GROUPS=2、GROUP_SHIFT=1，每两个连续八点组取一次最大值，形成覆盖 16 个频点的快照点，共 1024 点。cdf_bin 仍保留每个频点的累计功率，因此显示曲线的聚合不会降低带宽边界的定位粒度。

2.7.3 99% 带宽边界

阈值计算代码摘要

```
DIV_WAIT: if(div_done) begin
  lower <= quotient + (remainder != 0);
  upper <= rt - quotient;
  state <= SCAN;
end

SCAN: if(compare_valid) begin
  if(!found_low && rt != 0) begin
    if(cdf_bin[0] >= lower) begin
      lowq <= {{(16-LOGN){1'b0}},compare_addr,3'd0};
      found_low <= 1;
    end
    else if(cdf_bin[1] >= lower) begin
      lowq <= {{(16-LOGN){1'b0}},compare_addr,3'd1};
      found_low <= 1;
```

```
    end
    // 其余通道按相同方式判断
  end

  if(!found_high && rt != 0) begin
    if(cdf_bin[0] >= upper) begin
      highq <= {{(16-LOGN){1'b0}},compare_addr,3'd0};
      found_high <= 1;
    end
    else if(cdf_bin[1] >= upper) begin
      highq <= {{(16-LOGN){1'b0}},compare_addr,3'd1};
      found_high <= 1;
    end
    // 其余通道按相同方式判断
  end
end
```

设 ROI 内所有频点的总功率为 T，系统将总功率的 0.5% 和 99.5% 作为带宽边界：$L = \left\lfloor \frac{T}{200} \right\rfloor$，$H = T - \left\lfloor \frac{T}{200} \right\rfloor$。其中下限采用向上取整，上限扣除向下取整的尾部功率，与代码中 quotient 和 remainder 的处理一致。

扫描过程中，cdf_bin 保存按频率递增方向得到的累计功率。第一次满足$C(q) \geq L$的频点记为 lowq，第一次满足$C(q) \geq H$的频点记为 highq。因此，区间[lowq,highq]内的累计功率约占总功率的 99%，带宽计算为：$B_{99} = \frac{(q_{\text{high}} - q_{\text{low}})F_S}{16384}$。代码中的 found_low 和 found_high 用于锁定已经找到的边界，避免后续频点再次覆盖结果。离散频点可能使区间所含功率略高于目标比例，不能要求所有输入恰好得到 99%。

compare_addr 表示当前组的 11 bit 地址，与比较命中的 3 bit 通道号组成 14 bit 频点索引，再扩展为结果字段宽度。

固定分母除法代码摘录

```
udiv64 #(.DIVISOR_WIDTH(8)) div200 (
  .clk(clk), .rst(rst), .start(state==DIV_START),
  .numerator(rt), .denominator(8'd200),
  .busy(), .done(div_done), .divide_by_zero(),
  .quotient(quotient), .remainder(remainder)
);
```

分母 200 只需要 8 bit，除法器据此缩小内部除数和余数宽度，分子与商仍为 64 bit。该实现保留精确整数商和余数，不采用近似倒数，不改变带宽边界的阈值规则。

2.8 时域幅度

频域分析不能完整反映信号在时间方向上的幅度变化，因此，系统在 FFT 处理支路之外保留了一条原始时域测量支路，直接使用未加窗的 IQ 数据计算样本功率，并在固定 FFT 窗口和波形活动区间内分别统计峰值、能量和 RMS。

2.8.1 原始 IQ 功率计算与窗口能量累加

对于一个复数 IQ 样本：$x[n] = I[n] + jQ[n]$其数字功率定义为：$p[n] = I[n]^2 + Q[n]^2$。

原始 IQ 功率计算代码摘要

```
ii <= $signed(iq[15:0]) * $signed(iq[15:0]);
qq <= $signed(iq[31:16]) * $signed(iq[31:16]);

p  <= ii + qq;

wire [63:0] enext = energy + {32'b0, p};
wire [31:0] pnext = p > peak ? p : peak;
```

这段代码从 32 位 IQ 输入中提取 I 路和 Q 路数据，分别计算平方后相加，得到当前样本的数字功率 p。enext 用于累计当前窗口的总能量，pnext 用于更新窗口内的最大功率。能量使用 64 位寄存器，可以容纳多个样本功率连续累加；峰值仍保留为 32 位，因为它表示单个样本的最大功率。

这里采用原始 IQ 数据计算幅度，因此不会受到 Hann 窗系数的影响。FFT 支路中的窗函数主要用于降低频谱泄漏，而时域幅度测量需要反映原始采样值，所以两条支路必须分开。

2.8.2 固定窗口末点处理与统计结果输出

系统以 16384 个样本为一个固定窗口。窗口位置从 0 计数到 16383，当第 16384 个样本到达时，需要将该样本的功率计入能量和峰值统计，然后再清零寄存器开始下一个窗口。

如果在第 16384 个样本到达时直接保存 energy 和 peak，就会漏掉最后一个样本。因此代码使用 enext 和 pnext 保存包含当前样本后的下一状态值。

窗口末点纳入与结果锁存代码摘要

```
if(pv[1]) begin
  samples <= samples + 1;
  if(window_pos == 0)
    first_tick <= t1;

  window_pos <= window_pos + 1;

  if(window_pos == N-1) begin
    window_valid <= 1;
    window_data <= {window_id, first_tick, enext, pnext};

    window_id <= window_id + 1;
    energy <= 0;
    peak <= 0;
  end else begin
    energy <= enext;
    peak <= pnext;
  end
end
```

这段代码按照有效数据节拍统计样本数量，并在窗口第一个样本到达时记录该窗口的起始时间戳。当 window_pos 等于 16383 时，表示当前样本是本窗口的最后一个样本，模块将包含该样本的 enext 和 pnext 与窗口编号、起始时间一起写入 window_data，随后清零能量和峰值寄存器。其中 N=16384、LOGN=14，窗口位置使用 14 bit 计数；当前每个窗口覆盖 102.4 μs。

输出的窗口上下文会进入后续 FIFO，与 FFT 产生的频域结果按照窗口编号进行关联。这样，某个窗口的时域幅度和频域功率谱可以在结果组装阶段对应起来。

2.8.3 峰值、RMS 与定点输出

峰值与 RMS 代码摘要

```
PEAK_WAIT: if(sqrt_done) begin
  peak_result <= sqrt_root;

  if(is_burst) begin
    div_num <= br[95:32];
    div_den <= burst_length;
    state <= DIV_INT_START;
  end else begin
    sqrt_rad <= cr[95:32] << (32-LOGN);
    state <= RMS_START;
  end
end

RMS_START: state <= RMS_WAIT;

RMS_WAIT: if(sqrt_done) begin
  rms_result <= sqrt_root;
  state <= PUBLISH;
end
```

代码首先等待峰值开方完成，将结果保存为 peak_result。固定窗口的均方值为窗口能量除以 N，为得到 Q16.16 幅度输出，需要构造能量乘以 2 的 32 次方再除以 N 的开方输入。当前 N=16384，因此 sqrt_rad 由窗口能量左移 18 位得到。RMS 开方完成后保存 rms_result，再进入结果组装状态。

变长区间商与余数处理代码摘录

```
DIV_INT_WAIT: if(div_done) begin
  mean_int <= div_q;
  mean_rem <= div_r;
  div_num <= div_r << 32;
  state <= DIV_FRAC_START;
end
DIV_FRAC_START: state <= DIV_FRAC_WAIT;
DIV_FRAC_WAIT: if(div_done) begin
  sqrt_rad <= (mean_int << 32) | div_q;
  state <= RMS_START;
end
```

波形活动区间长度不固定,第一次除法使用总能量除以实际样本数，保存整数商和余数；第二次将余数左移 32 位后仍除以该长度，形成均方值的小数部分。两部分组合后送入整数开方，避免仅保留整数商造成的小数损失。峰值和 RMS 共用开方单元,整数开方取下整，与窗乘积的收敛舍入规则分别处理。

数学单元比较与余数更新代码摘录

```
wire [DIVISOR_WIDTH:0] rnext =
  {remnant,dividend[63]};
wire [DIVISOR_WIDTH+1:0] difference =
  {1'b0,rnext} - {2'b0,divisor};

if(!difference[DIVISOR_WIDTH+1]) begin
  remnant <= difference[DIVISOR_WIDTH-1:0];
  quotient <= (quotient<<1) | 1;
end else begin
  remnant <= rnext[DIVISOR_WIDTH-1:0];
  quotient <= quotient<<1;
end
```

扩展减法的最高位用于判断是否借位,比较和余数更新由同一次减法完成。除法器按除数范围保存余数，开方器也根据余数上界设置内部位宽，从而缩短运算路径；输入与输出位宽及精确整数运算规则保持不变。

2.9 波形长度检测与完整性标记

系统根据时域 IQ 功率检测波形活动区间，工程中提供门限检测和数字零检测两种方式。门限检测适用于存在背景功率的输入，其结果受噪声、门限和确认参数影响；数字零检测适用于静默样本严格为 I=0、Q=0 的测试数据。

2.9.1 滑动功率与双门限判断

门限检测代码摘要

```
history[hp] <= p;
hp <= hp + 1;
if(filled < 16)
  filled <= filled + 1;

power_delta <= $signed({1'b0,p}) -
               $signed({1'b0,(filled == 16 ? history[hp] :
32'd0)});

p_delta <= p;
n_delta <= samples;

if(detector_valid[0]) begin
  sliding <= sliding + {{3{power_delta[32]}},power_delta};
  p_sum <= p_delta;
  n_sum <= n_delta;
end

if(detector_valid[1]) begin
  above_on <= sliding > ton;
  below_off <= sliding < toff;
  p_detector <= p_sum;
  n_detector <= n_sum;
end
```

使用 16 点滑动窗口计算局部功率。每接收一个新样本，就把当前功率加入滑动和，同时减去 16 个样本之前的功率，因此滑动功率可以表示为：$S[n] = \sum_{i=0}^{15} p\,[n-i]$。

power_delta 使用有符号数保存新旧功率的差值，避免滑动窗口更新时出现下溢或溢出。经过流水线延迟后，系统将滑动功率与起始门限 ton 和结束门限 toff 比较。只有当 sliding > ton 时才认为信号可能开始，只有当 sliding < toff 时才认为信号可能结束。起始和结束使用不同门限，可以避免信号在临界值附近波动时反复触发。

2.9.2 起始确认、活动状态与结束确认

起止确认阈值与状态转换代码摘录

```
on_last <= kon==0 ? 16'd0 : kon-16'd1;
off_last <= koff==0 ? 16'd0 : koff-16'd1;

// START_CAND 分支：连续满足起始条件
if(above_on) begin
  benergy <= benergy+p_detector;
  if(p_detector>bpeak) bpeak <= p_detector;
  confirm <= confirm+1;
  if(confirm>=on_last) begin
    if(burst_limit_reached)
      emit_burst(n_detector+1,benergy+p_detector,
        p_detector>bpeak?p_detector:bpeak,32'h400);
    else state <= ACTIVE;
  end
end else state <= IDLE;

// END_CAND 分支：确认结束或合并暂存尾段
if(below_off && confirm>=off_last)
  emit_burst(be,benergy,bpeak,0);
else if(burst_limit_reached)
  emit_burst(n_detector+1,benergy+tail_energy+p_detector,
             merged_peak,32'h400);
else if(below_off) begin
  confirm <= confirm+1;
  tail_energy <= tail_energy+p_detector;
  if(p_detector>tail_peak) tail_peak <= p_detector;
end else begin
  benergy <= benergy+tail_energy+p_detector;
  bpeak <= merged_peak;
  state <= ACTIVE;
end
```

系统先处于 IDLE,满足起始条件后记录候选起点、能量和峰值；kon=1 时直接进入 ACTIVE，否则进入 START_CAND。只有连续满足 kon 次条件才确认活动区间，失败则取消候选。

在 ACTIVE 状态下，系统持续累加活动区间的能量并更新峰值。当功率低于结束门限时，模块会进入 END_CAND，连续确认 koff 个样本。若低功率状态持续满足条件，则输出活动区间；若信号重新升高，则将暂存的尾段重新合并到活动区间中。这样可以保留信号内部较短的低功率间隔，避免把一个连续信号错误地分成多个区间。活动区间长度采用半开区间表示：$M = end_sample - start_sample$。当 koff=1 时直接按当前候选边界结束；确认成功时排除确认尾段，信号恢复时才将尾段能量和峰值合并回去。

最大长度判断预计算代码摘录

```
always @(posedge clk) begin
  maximum_previous <= max_burst-32'd1;
  if(rst) burst_limit_reached <= 0;
  else if(detector_valid[2] && state==IDLE && above_on)
    burst_limit_reached <= max_burst<=2;
  else if(detector_valid[2] && state!=IDLE)
    burst_limit_reached <=
      next_burst_length>=maximum_previous &&
      next_burst_length!=32'hffffffff;
  else burst_limit_reached <= next_burst_length>=max_burst;
end
```

next_burst_length 表示包含下一检测样本后的区间长度。burst_limit_reached 将长度比较提前寄存，在原样本节拍内决定是否输出超长结果。达到限制时结果带 DETECTOR_TIMEOUT 标志；采集结束但正常边界尚未确认时，带 CAPTURE_TRUNCATED 标志输出已观察区间。

2.9.3 数字零检测与完整性标志

数字 0 检测代码摘要

```
always @(posedge clk) begin
  gap_last <= gap_min-16'd1;
  gap_penultimate <= gap_min-16'd2;
```

```
  length_last <= max_burst-32'd1;
end
always @(posedge clk) begin
  if(rst || !enable) gap_confirm <= gap_min<=1;
  else if(valid)
    gap_confirm <= gap_last==0 ||
      (power==0 && zeros>=gap_penultimate);
  else gap_confirm <= zeros>=gap_last;
end

// valid 且 active 时的区间结束与分段判断
if(power==0 && gap_confirm) begin
  if(has_nonzero)
    emit(last_nonzero+1,energy,peak,base_flags);
  active <= 0;
  length <= 0;
  has_nonzero <= 0;
end else if(length>=length_last) begin
  emit(sample_index+1,energy_next,peak_next,
       base_flags | DETECTOR_TIMEOUT);
  start_sample <= sample_index+1;
  start_unconfirmed <= 1;
  length <= 0;
  energy <= 0;
  peak <= 0;
  has_nonzero <= 0;
end else begin
  length <= length+1;
  energy <= energy_next;
  peak <= peak_next;
  if(power!=0) has_nonzero <= 1;
end
```

当功率为零时，zeros 连续计数；当功率非零时，计数清零并更新最后一个非零样本位置 last_nonzero。当连续零样本数量达到 gap_min 后，系统认为活动区间结束，结束位置取最后一个非零样本的下一个位置，因此用于结束确认的零样本不会被计入有效区间，而活动区间内部较短的零间隔仍会保留在长度和 RMS 分母中。gap_confirm 根据下一次计数关系提前寄存，处理有效样本暂停及 gap_min=1 的情况。零间隙确认与长度限制同拍发生时，优先按确认的正常结束边界处理。

有限采集结束与完整性标志代码摘录

```
wire [31:0] base_flags = DIGITAL_ZERO |
  (start_unconfirmed ? START_UNCONFIRMED : 0);

if(finish && !valid && active) begin
  if(length!=0)
    emit(observed_end,energy,peak,
         base_flags | CAPTURE_TRUNCATED);
  active <= 0;
  length <= 0;
  has_nonzero <= 0;
end
```

DIGITAL_ZERO 标识数字零模式，START_UNCONFIRMED 表示起点前未观察到足够零间隙。若停止采集时结束间隙尚未确认，按 observed_end 闭合区间并置 CAPTURE_TRUNCATED，此时未确认的尾零仍属于观察区间。达到最大长度产生的分段带 DETECTOR_TIMEOUT 标志。

2.10 结果记录与上位机读取

结果构建模块按照窗口编号关联时域统计与频域结果，生成 1024 bit 频域记录；活动区间统计独立形成 512 bit 突发记录。两类记录分别写入容量为 128 条的环形缓冲区，保存测量数值、编号及完整性标志，供 PS 读取。

记录按 32 bit 分次写入，完成整条记录后才更新生产者计数。PS 通过 GP0 读取已发布记录，并以 ACK 更新消费者计数，释放已读空间。ACK 先捕获请求并寄存合法性判断，再执行一次消费者更新，同时维护计数回绕关系；缓冲区满或写入忙时增加拒绝计数，非法 ACK 不推进消费者，错误状态供软件检查。

3 功能仿真结果

3.1 仿真环境与方法

本项目采用 Vivado 2026.1 的 XSim 进行行为级仿真，分别验证数学单元、测量模块、接口控制和完整分析核心。完整核心的源时钟与 FFT 时钟均为 160 MHz，测试覆盖零输入、正负复单音、突发单音、两种 QPSK 向量、跨窗口短突发及负满幅直流，每种输入分别使用矩形窗和 Hann 窗，共 16 组用例。每组输入 65536 对 IQ，对应四个 16384 点窗口。各单元测试使用自身测试平台时钟，波形时间轴与完整核心时钟分别解释。

FFT 输出采用与当前 16384 点、24 bit 数据、18 bit 旋转因子及缩放配置匹配的 AMD 位精确模型作为参考；时域能量、幅度、活动区间及频谱参数由独立整数参考计算和测试断言检查。验证除观察波形，还比较输出字段、有效时序、窗口数量和错误状态。共享功率谱、Hann 预处理和时域控制优化另与参考实现进行等价性比较，提交和 ACK 控制则检查事务捕获、取消、计数回绕及单次执行。

3.2 FFT 输出与频率换算

图 3-1 和图 3-2 展示 FFT 输入、输出索引与窗口结束关系。图 3-1 中，预处理窗内计数在 16383 后回到 0，输入结束标志经过四级流水，与数据对齐后送至 FFT；图 3-2 中，FFT 在末点输出结束标志，随后进入下一窗口。

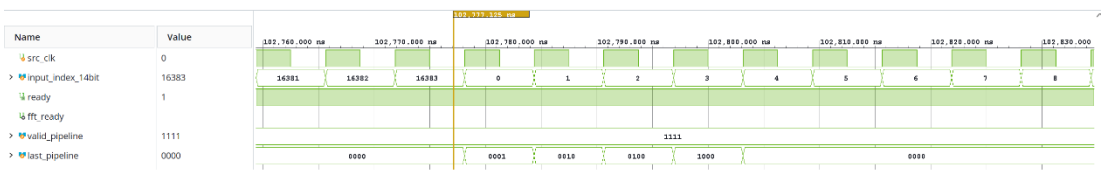


图 3-1 FFT 输入窗口索引与末点标志

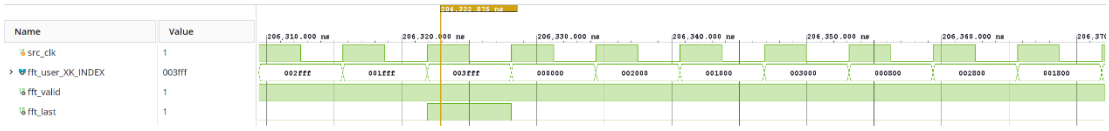


图 3-2 FFT 输出末点标志与相邻窗口边界

测试按照 XK_INDEX 查找对应参考值，逐点比较复数结果，并检查每窗有效输出恰为 16384 点。16 组核心用例共完成 1048576 个复数 FFT 点比较，产生 64 条频域记录，归档结果为 PASS。在当前测试输入下，FFT 配置、加窗定点处理及输出索引关系与位精确参考一致。

频率换算另由结果构建单元覆盖全部 16384 个 bin，检查正负频率、带宽中心、上下边界及整数舍入。当前频率间隔为 9.765625 kHz，160 MSPS 下正、负 Fs/4 单音分别对应 +40 MHz、−40 MHz。

当前 Hann 预处理采用四分之一周期系数存储和重构。图 3-3 对照当前实现与参考实现，输入空拍和后级暂停时，两路 ready、valid 及数据保持一致。等价性测试覆盖全部 Hann 索引、符号极值、空拍、后压和复位，共完成 98333 次输出比较，结果为 PASS。

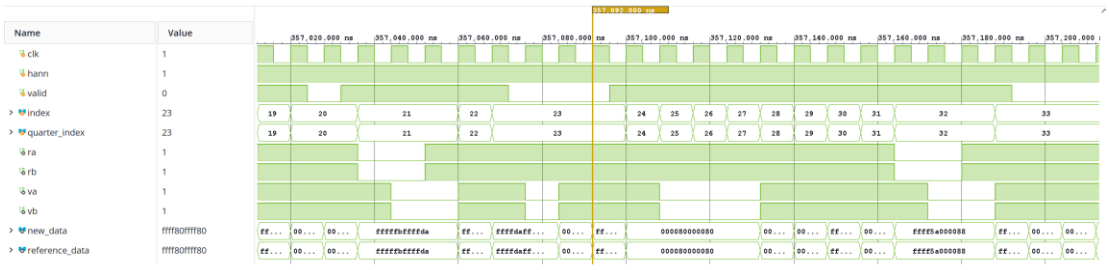


图 3-3 Hann 预处理重构与参考实现比较

3.3 占用带宽与频谱边界验证

图 3-4 展示累计功率、阈值和实际上下边界的对应关系。case28

的升序频点 q=2000、2008 的功率均为 10000，总功率为 20000。下、上阈值分别为 ceil(20000/200)=100 和 20000−floor(20000/200)=19900；累计功率首次达到这两个阈值的位置分别为 q=2000 和 q=2008，与硬件记录一致。在 160 MSPS、16384 点配置下，八个频点间隔对应带宽 78.125 kHz。

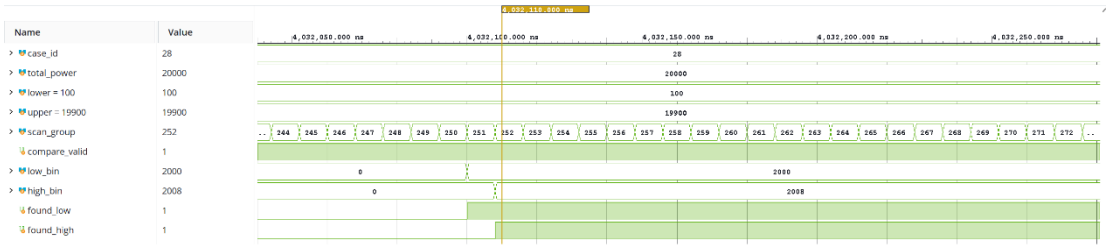


图 3-4 累计功率阈值与上下边界定位

为检查八路扫描是否保留组内频点粒度，测试将两个边界依次移动至 lane0～7。图 3-5 列出 case28～35 的参考值与实际输出，八组上下边界均一致，即边界定位未被简化为八点分组的首、末位置。测试还覆盖零谱、单 bin、频带边缘、ROI 及扫描复位等情况，共完成 36 帧和 6144 个快照字检查，结果为 PASS。频谱单元测试时钟为 125 MHz。

八路扫描保留单频点边界定位

用例	Lane	参考下界	实际下界	参考上界	实际上界	结果
28	0	2000	2000	2008	2008	一致
29	1	2001	2001	2009	2009	一致
30	2	2002	2002	2010	2010	一致
31	3	2003	2003	2011	2011	一致
32	4	2004	2004	2012	2012	一致
33	5	2005	2005	2013	2013	一致
34	6	2006	2006	2014	2014	一致
35	7	2007	2007	2015	2015	一致

图 3-5 八路边界定位参考值与实际输出

带宽边界使用所选分析范围内的完整频点功率，按累计功率首次达到整数阈值的位置确定，再以两端频点中心之差计算带宽。

当前频谱测量采用一份完整功率谱和延迟写入队列，窗口统计信

息另行保存。图 3-6 中，上一窗口扫描期间保护尚需读取的功率数据，新到达且暂不能覆盖的数据进入队列；保护解除后排出队列并回写。测试按相同输入与参考双谱存储实现比较结果、快照和输出周期，共检查 680069 个周期，结果一致。队列最高占用为 2123 条，低于 2560 条容量，该数值为本组测试条件下的观测结果。

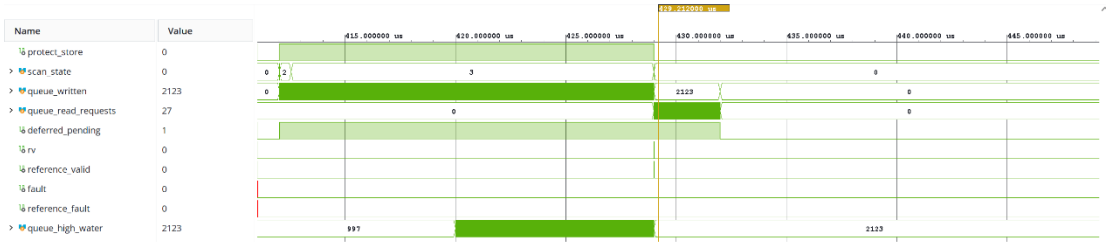


图 3-6 共享功率谱保护与延迟写入队列

3.4 幅度与活动区间长度

幅度验证检查原始 IQ 平方和、窗口末样本计入、峰值搜索及 Q16.16 结果转换。图 3-7 和图 3-8 采用结果构建单元的固定参考用例：输入峰功率为 67108864，16384 点窗口能量为 1099511627776，理论峰值和 RMS 均为 8192。两项实际输出均为 0x20000000，除以 2^16 后得到 8192，与参考计算一致。图中 100 MHz 为该单元测试时钟。

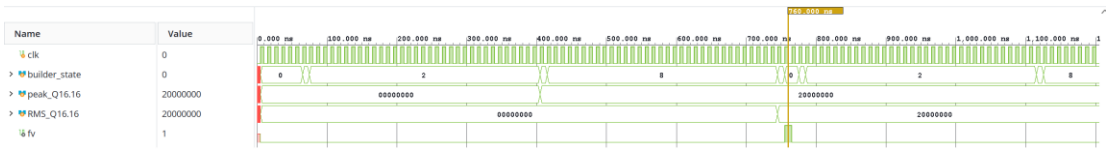


图 3-7 固定幅度用例的定点结果波形

固定窗口幅度参考计算与硬件结果

项目	参考 / 输入	实际结果
窗口点数	16384	16384
峰值功率	67108864	67108864
窗口能量	1099511627776	1099511627776
峰值 Q16.16	0x20000000	0x20000000
RMS Q16.16	0x20000000	0x20000000
幅度（数字码值）	8192	峰值 8192 / RMS 8192

图 3-8 固定幅度用例的参考计算与实际输出

活动区间检测分别验证门限模式和数字零背景模式。图 3-9 中，滑动功率下降后进入候选结束状态 END_CAND，经连续确认后提交记录，结束位置保留为 26639，输出区间为 [2048,26639)。输入矩形包络的设计终点为 26624，15 点差异来自 16 点滑动功率的响应。检测长度为 24591 点，在 160 MSPS 下对应 153.69375 μs，不能与数字零模式的非零支撑区间混用。



图 3-9 门限结束确认与候选结束位置

图 3-10、图 3-11 比较数字零模式 G=32 时的两种间隔。内部 31 个连续零样本不足以确认结束，两段活动合并为 [32,68)；内部 32 个连续零样本满足确认条件，分别输出 [32,34) 和 [66,69)。区间采用左闭右开定义，图中 last_nonzero 为最后一个非零样本编号，记录终点为其加 1。



图 3-10 31 点短间隔下的突发合并

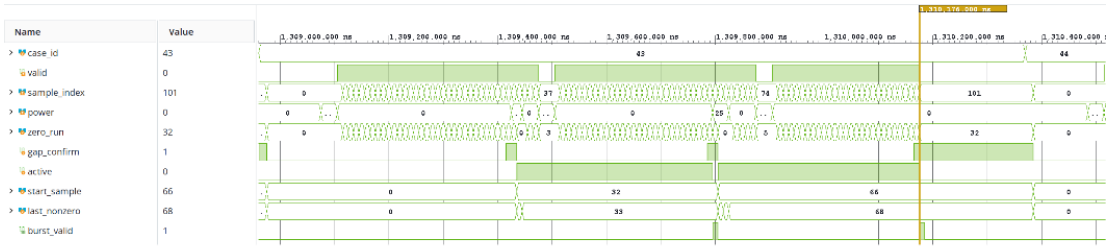


图 3-11 32 点确认间隔下的突发分离

数字零单元还覆盖结束边界、截断、强制分段及有效样本空拍，共完成 344 组用例、3469946 个输入样本和 225699 条突发记录比较，结果为 PASS。

针对时域控制条件寄存化，等价性测试覆盖不同 kon、koff、暂停、长度限制及结束条件，共检查 274071 个周期、224 个突发和 16 个窗口，测试周期为 6.666 ns。数学单元另以 100 MHz 测试时钟完成 10220 组向量检查，覆盖精确整数开方、除法、零分母及运算中复位。两类测试均通过，已测试条件下的输出与参考一致。

3.5 接口控制与时延

图 3-12 和图 3-13 展示 DMA 异常流结束保护和窗口边界切换。图 3-12 中，输入提前出现末字标志后，提交检查结束，装载活动撤销，loader_done 保持为 0；状态地址 0x1D4 的有效读回值为 0xC，表明错误块未作为正常完成块提交。图中的 loader_errors=0x11 为独立错误字段，与状态读回值分别解释。

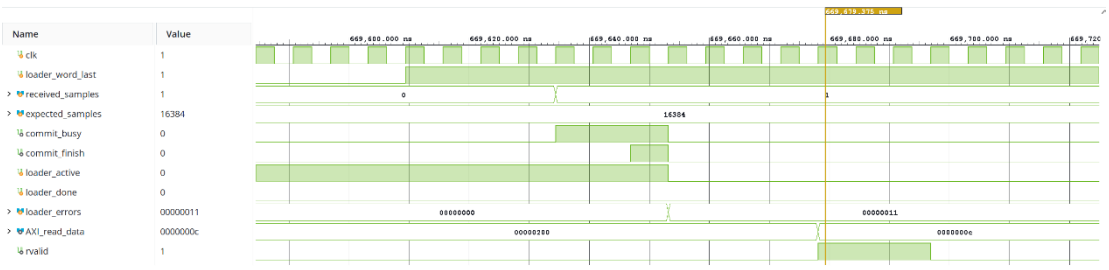


图 3-12 DMA 装载异常末字拒绝

图 3-13 中，已准备的新块等待窗口边界，在 issued 从 180223 增至 180224，即 11×16384 的边界，活动 Bank 从 0 切换为 1，block_id 从 11 更新为 22，pending_valid 同时清零。回放选择与同步 RAM 读延迟一同对齐，避免切换处混入错误 Bank 的样本。

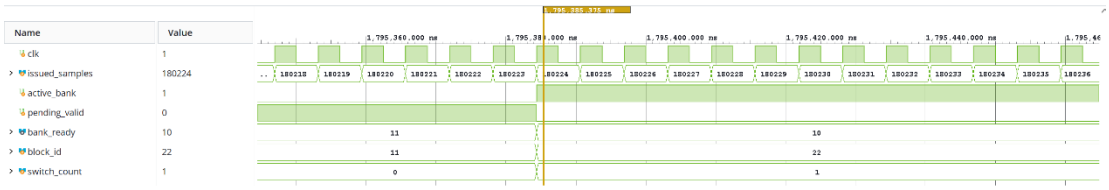


图 3-13 16384 点窗口边界上的双 Bank 切换

图 3-14 展示分阶段提交过程。事务开始时捕获输入，随后进行检查、路由和完成处理，外部 block_id 改变不影响已捕获的数据。正常事务发布目标 Bank，错误事务只完成检查而不发布 Bank。独立提交测试共完成 64 次事务及三个取消阶段，检查元数据原子更新、字节写使能、载荷捕获和单次完成，结果为 PASS。

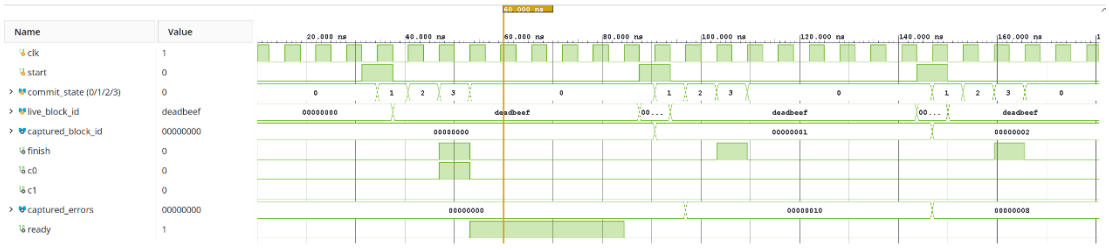


图 3-14 分阶段提交与异常事务拒绝

图 3-15 展示 ACK 检查与提交关系。合法性检查结果先寄存，随后 consumer 从 0 更新为 1；同一事务只执行一次，后续 producer 变化不改变已捕获的决定。相关测试同时覆盖计数回绕、未来序号和

consumer 边界。

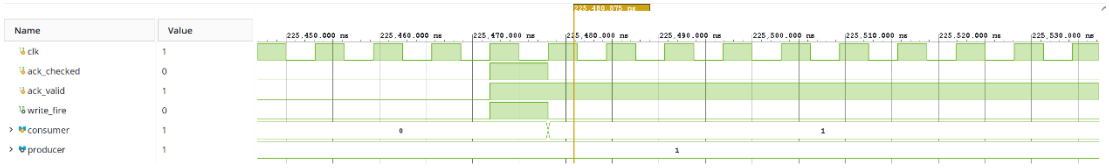


图 3-15 ACK 检查结果寄存与单次提交

图 3-16、图 3-17 按统一仿真时基整理首输入、FFT 输出、窗口元数据锁存、扫描和分析完成事件。以 case0 的首窗口为例，首样本消费事件位于 0.334375 μs，analysis_done 位于 220.109375 μs，两者之差为 219.775 μs。64 个窗口的归档核心最大分析时间为 35164 个源时钟周期，在 160 MHz 下对应同一数值。

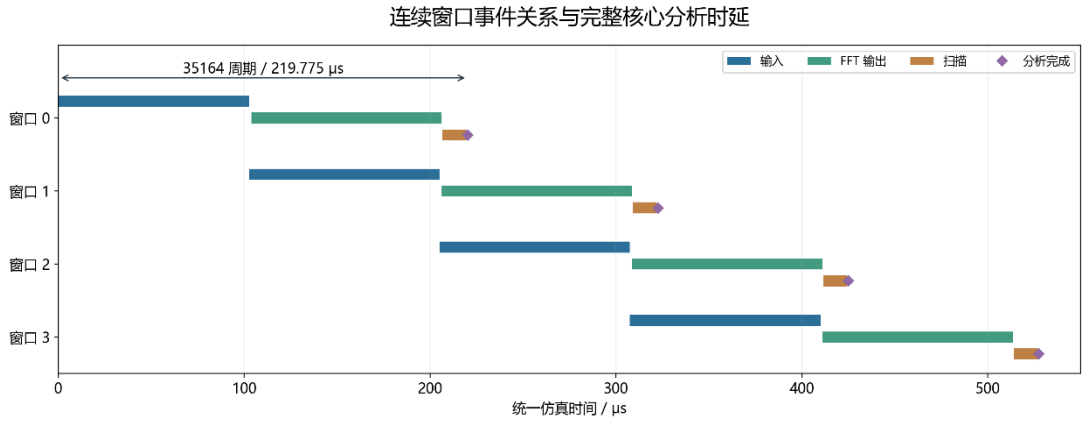


图 3-16 连续窗口事件关系与完整核心分析时延

首窗口事件时间与延迟定义

事件	绝对时间 / μs
首样本消费	0.334375
末样本消费	102.728125
首频点输出	103.934375
末频点输出	206.328125
窗口元数据锁存	206.353125
首组扫描请求	206.778125
末组边界比较	219.615625
扫描结果生成	219.621875
分析结果生成	220.109375
测试平台观察到记录	220.115625

图 3-17 首窗口事件时间与分析时延计算

4 实板测试结果

4.1 测试平台

实板测试采用 Zybo Z7-20，器件为 XC7Z020CLG400-1，开发工具为 Vivado 2026.1，开发板采用 USB 供电。PC 经以太网向 PS 装载 I、Q 各 16 bit 的数字基带数据，PS 通过 DDR 缓冲和 AXI DMA 将数据送入 PL 非活动 Bank；活动 Bank 以 160 MSPS 独立回放，测量记录进入 PL 结果环形缓冲后，由 PS 读取并回传上位机。每个 IQ Bank 最多保存 32768 对 IQ，合法装载长度为 16384 或 32768 对 IQ，分别对应一个或两个分析窗口。本节所列测试采用 32768 对 IQ 的数据块。测试连接、当前数据路径和板卡身份分别如图 4-1、图 4-2 所示。

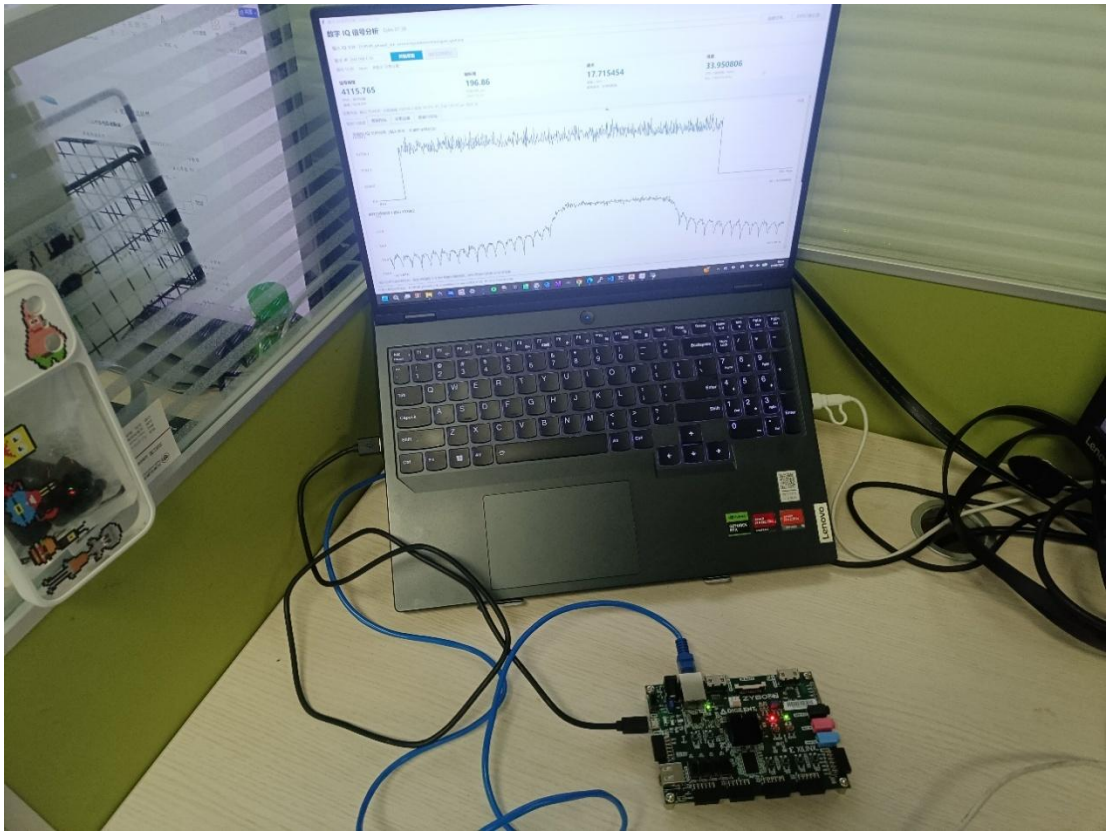


图 4-1 实板连接示意

本次实板读取到的硬件版本为 0x0001000b，构建 ID 为 4757a33c02eed50a403db87d09cb091f。当前频谱后处理采用单份完整功率谱和延后写入队列，并保留双窗口元数据，与输入侧 IQ 双 Bank

分别承担不同的缓存任务。

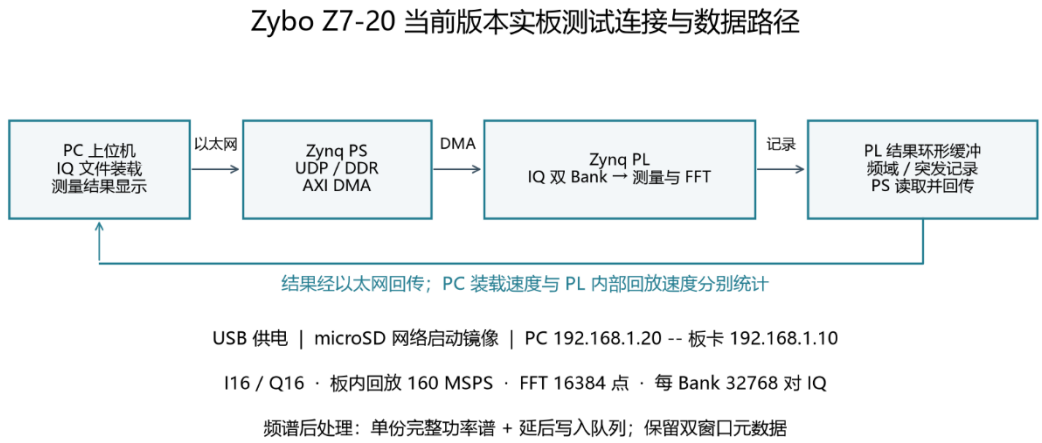


图 4-2 当前版本测试连接与数据路径

4.2 资源与时序

布局布线后的资源利用率和静态时序结果如图 4-3 4-4 所示。当前实现使用 LUT 22708 个，占器件资源的 42.68%；FF 28413 个，占 26.70%；DSP 49 个，占 22.27%。

Summary

Resource	Utilization	Available	Utilization %
LUT	22708	53200	42.68
LUTRAM	5740	17400	32.99
FF	28413	106400	26.70
BRAM	128	140	91.43
DSP	49	220	22.27
MMCM	1	4	25.00

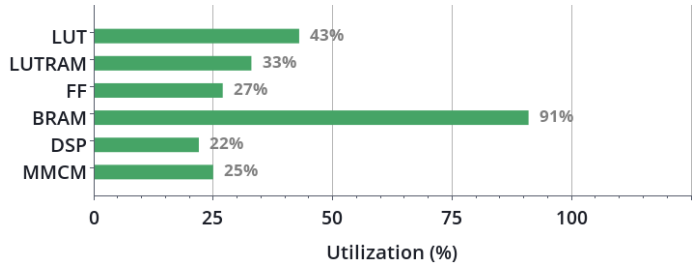


图 4-3 当前实现的资源利用率

在源时钟与 FFT 时钟均为 160 MHz 的约束下，最差建立时间裕量为 +0.007 ns，最差保持时间裕量为 +0.050 ns，建立与保持违例端点均为零，内部未约束端点和 critical CDC 检查项均为零。最差建立路径位于频谱累计分位边界相关逻辑，由 cdf_bin_reg[5][3] 到 highq_reg[0] 的使能端，数据路径延迟为 5.827 ns，其中布线延迟为 3.651 ns，约占 62.658%。

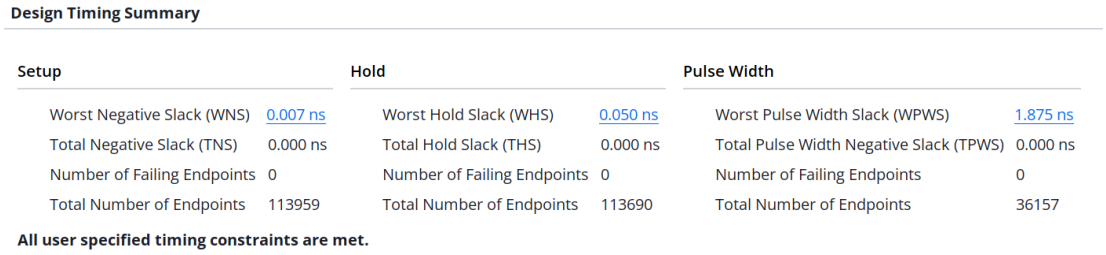


图 4-4 当前实现的时序裕量

4.3 测量结果

4.3.1 基础指标

根据赛题要求，系统需要测量信号幅度、帧长度、频点和带宽，数字 IQ 位宽不低于 12 bit，数据速率不低于 100 MSPS，FFT 点数不低于 8192 点，分析处理时间不超过 2 ms。当前系统采用 I、Q 各 16 bit、160 MSPS 板内回放和 16384 点 FFT。为覆盖不同信号形态，选取窄带单音、跨窗口突发和宽带 OFDM 三组数字 IQ 信号进行实板测量，并用当前定点参考检查输出记录。幅度以数字码值表示，帧长度按检测到的活动包络解释。

信号 1 采用 Hann 窗，实板测量结果如图 4-5 所示。测得信号幅度 RMS 和峰值均为 8192.000 码，峰频率为 40.000000 MHz，99% 占用带宽为 0.019531 MHz，带宽中心为 40.000000 MHz。该信号用于说明系统对窄带信号的幅度、频点和带宽能够给出数值测量结果。界面中的 204.80 μs 对应 32768 对 IQ 的有限回放区间，并带有停止截断标志，不能将其作为已经确认结束的完整突发长度；单个 16384 点 FFT 窗口对应的时间为 102.4 μs。

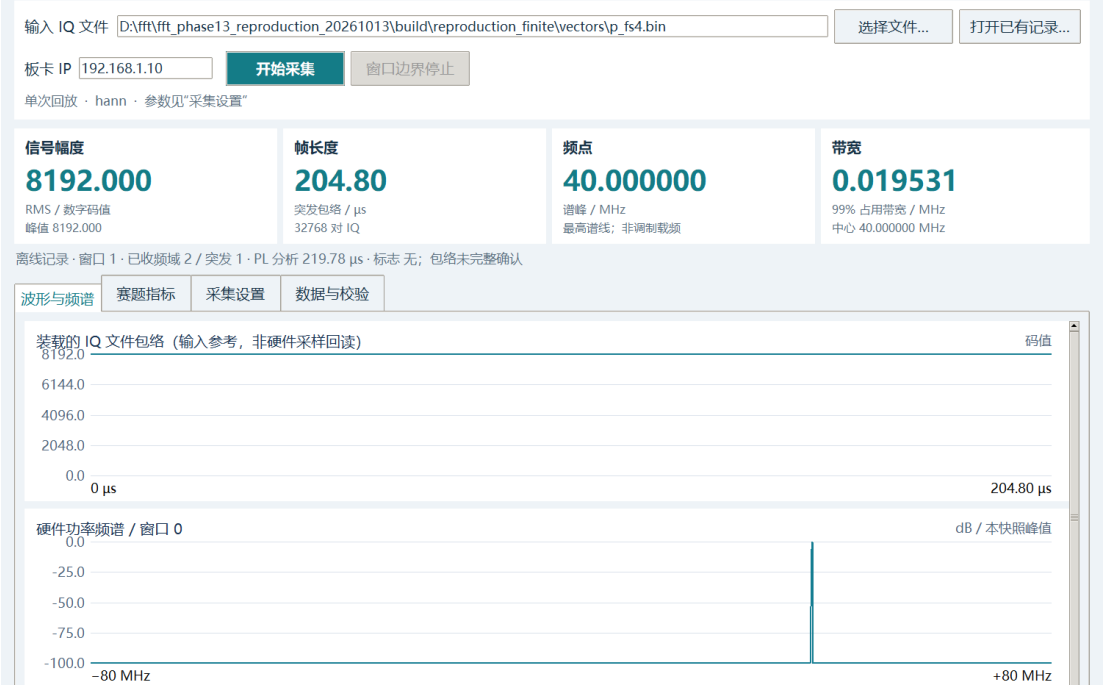


图 4-5 窄带单音实板测量（Hann 窗，停止截断）

与此同时，本次实板数值矩阵覆盖全零、负满幅、低幅度、不同单音频率、双音、调制信号、窗口边界突发及宽带信号等输入，共完成 79 项检查。频域、突发、标志、序号、时间戳及可用频谱快照按照各自的定点参考进行核对，检查结果为 PASS。

信号 2 采用数字零背景检测和矩形分析窗，实板测量结果如图 4-6 所示。检测到的活动区间为 [16128,16640)，跨越第 16384 点的分析窗口边界，长度为 512 对 IQ。按照 160 MSPS 回放速率换算，512/160 MHz=3.200 μs，与界面显示一致。两个分析窗口均有频域记录，活动区间合并形成一条完整突发记录，说明波形测长不局限于单个 FFT 窗口。



图 4-6 跨窗口数字零背景突发实板测量

信号 3 采用宽带 OFDM 输入和 Hann 窗，实板测量结果如图 4-7 所示。图中窗口 0 的 RMS 为 5811.717 码，峰值为 20155.028 码，峰频率为 −75.537109 MHz，99% 占用带宽为 158.515625 MHz，带宽中心为 −0.009766 MHz。顶部频域数值与下方频谱快照均对应窗口 0，界面显示的门限突发长度为 26127 对 IQ，即 163.293750 μs，按活动包络长度解释。该结果说明系统能够对接近 160 MHz 数字基带频率范围的宽带信号进行带宽统计。



图 4-7 宽带 OFDM 实板测量（Hann 窗）

为避免仅用单个 OFDM 向量说明宽带能力，当前版本还完成了包含不同带宽、调制方式、随机种子、幅度和频移条件的 272 项宽带实板检查。其中主组满带子集的 99% 占用带宽范围为 158.154297～158.593750 MHz，如图 4-8 所示，当前频点间隔为 160 MHz/16384=9.765625 kHz，带宽边界按完整功率谱的累计功率分位确定。

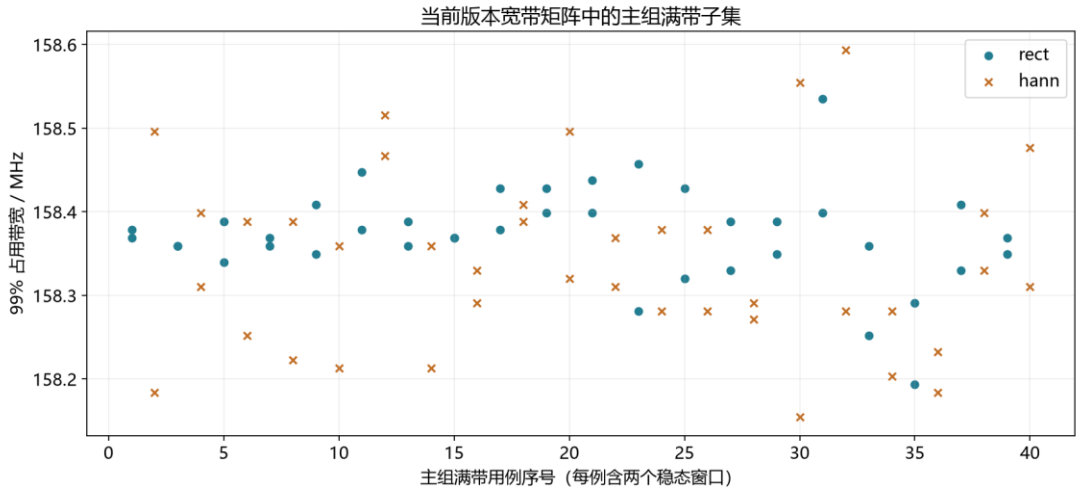


图 4-8 宽带矩阵主组满带子集

4.3.2 噪声条件与测试边界

含噪测试采用数字向量生成的 SNR 参数，实际检测门限由测试配置明确写入，并记录在采集文件中。检查覆盖 0 dB、10 dB 和 20 dB 三组条件，每组使用三个噪声种子，每个种子包含两个真值突发。10 dB 和 20 dB 两组均实现 6/6 正常匹配，未出现漏检或误检；0 dB 组的六个真值突发均未检出。该结果说明当前检测参数在所测 10 dB 和 20 dB 条件下能够完成包络测长，同时在 0 dB 条件下存在明确的漏检边界。

图 4-9 给出了 10 dB、噪声种子 91001 的实板结果，统计与误差见图 4-10。顶部频域数值与下方频谱快照均对应窗口 0，长度栏显示独立记录的最后一个突发。图中最后一个突发的真值区间为 [16384,24576)，实测区间为 [16389,24591)，真值长度为 8192 点，实测长度为 8202 点，即 51.2625 μs。长度误差为 10 点，即 0.0625 μs；起点和终点误差分别为 5 点和 15 点，均在 32 点边界容差内。该例实际门限 ton/toff 为 415107548/207553774，确认参数 kon/koff 为 8/32。

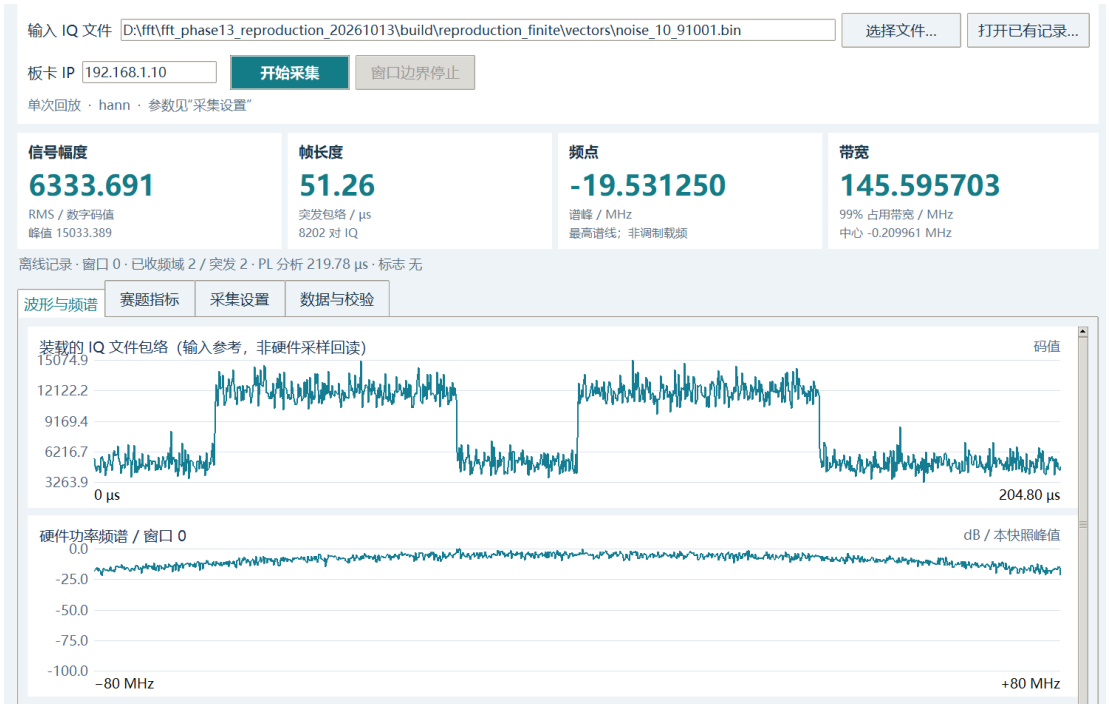


图 4-9 10 dB 含噪突发实板测量

当前含噪测试结果

项数值矩阵的含噪子集
采样率 160 MSPS；Hann 窗；每个种子含 2 个真值突发。

SNR / dB	种子数	真值突发	正常匹配	漏检	误检
0	3	6	0	6	0
10	3	6	6	0	0
20	3	6	6	0	0

```
10 dB，种子 91001，图中最后一个突发：
truth: [16384, 24576)   measured: [16389, 24591)
length_samples: 8202   length_us: 51.2625
start_error_samples: 5   end_error_samples: 15
length_error_samples: 10   length_error_us: 0.0625

applied_detector: ton=415107548, toff=207553774, kon=8, koff=32
```

图 4-10 含噪检出统计与边界误差

4.4 动态切换与稳定性

为验证数据块更新不会中断板内分析链路，本节在前述实板连接和测量方法的基础上，进一步测试 IQ 双 Bank 的连续回放、窗口边界切换及结果记录完整性。测试采用 160 MSPS 板内回放和 16384 点 FFT，每个数据块包含 32768 对 IQ。上位机依次准备单音、双音、chirp、QPSK、OFDM 和噪声六类数字数据，经 UDP 写入 PS 侧 DDR，并由 AXI DMA 装载到非活动 Bank。数据块通过 CRC、编号和长度检查后才能提交，活动 Bank 继续供数，新 Bank 仅在完整装载且当前 16384 点窗口结束后被激活。

图 4-11 展示了当前实板记录的详细字段，包括 RMS、峰值、活动包络长度、峰频率、99% 占用带宽、PL 分析延迟、结果发布延迟及完整性检查状态。当前最大分析延迟为 219.775 μs，最大结果入环延迟为 219.993750 μs。



图 4-11 实板测量记录与分析时延

图 4-12 为本次 1000 次动态 Bank 切换与异常输入拒绝验证结果。测试请求并完成 1000 次切换，输入覆盖 single、dual、chirp、QPSK、OFDM 和 noise 六类信号，测试范围包括 PC 写入 DDR、AXI DMA 装载非活动 PL Bank，以及活动 Bank 按 160 MSPS 回放，记录状态为 PASS。同时，对数据包 CRC 损坏、offset 乱序、未完整提交和写入活动 Bank 四类异常进行注入，分别得到 REJECTED 或 REJECTED_BY_FIRMWARE，说明异常事务被拒绝，没有进入正常切换流程。

1000 次动态切换验证结果

```
status: PASS
started_at: 2026-10-14T19:24:55.359669+08:00
finished_at: 2026-10-14T19:25:46.177302+08:00
build_id: 4757a33c02eed50a403db87d09cb091f
sample_rate_hz: 160000000   fft_length: 16384
samples_per_bank: 32768
switches_requested: 1000   completed_transitions: 1000
signals: single, dual, chirp, qpsk, ofdm, noise

packet_crc_corruption: REJECTED
out_of_order_offset: REJECTED
incomplete_commit: REJECTED
active_bank_write: REJECTED_BY_FIRMWARE

effective_average_upload_mbps: 21.880890
hardware_max_latency_us: 219.775
hardware_max_publish_latency_us: 219.99375
```

图 4-12 1000 次动态切换与异常输入拒绝

停止采集后，系统先进入排空状态，等待已经进入流水线的样本、FFT 输出、频谱扫描和突发记录处理完成，再读取最终计数。图 4-13 显示，1000 次切换测试中，发出样本数、接受样本数、FFT 输入样本数和 FFT 输出样本数均为 7967260672，完成 FFT 窗口数为 486283；输入拒绝、结果队列拒绝、输入下溢和 DMA 错误均为零，频域记录序号间隙和 UDP 缺包数也为零。四项样本计数相等，且窗口数与 16384 点 FFT 长度对应，说明停止排空后没有遗留未处理样本。

1000 次动态切换完整性验证

```
status: PASS   completed_windows: 486283

STOP 完成并排空后读取 diagnostics:
issued_samples: 7967260672
accepted_samples: 7967260672
fft_input_samples: 7967260672
fft_output_samples: 7967260672
fft_output_windows: 486283
input_rejected: 0
result_queue_rejected: 0
input_underreads: 0

statistics:
frequency_records: 486283
frequency_id_gaps: 0
udp_missing_packet_count: 0
recovered_malformed_packets: 0

DMA errors: 0
stream_errors: 0
write_rejected: 0
```

图 4-13 1000 次切换后的停止排空与计数守恒

4.5 指标汇总

数字基带 IQ 信号位宽	IQ 信号数据速率	FFT 点数	分析处理时间	99%占用带宽
I、Q 各 16 bit	160 MSPS	16384	219.775 μs	158.154297～158.593750 MHz

5 创新点

亮点一：八路并行扫描，带宽边界精确到单个频点。 本项目将 16384 点功率谱分为 2048 组，每拍读取八个连续频点，通过两项、四项、八项的平衡前缀流水计算累计功率。组间反馈路径只保留一级 64 bit 加法，避免八点串行累加形成长组合路径。阈值判断保留组内全部频点，而不是只比较每组总功率，缩短扫描时间的同时，不降低带宽边界的定位粒度。图 5-1a 展示了组内前缀与组间累计的衔接方式，阈值比较仍保留到单个频点。

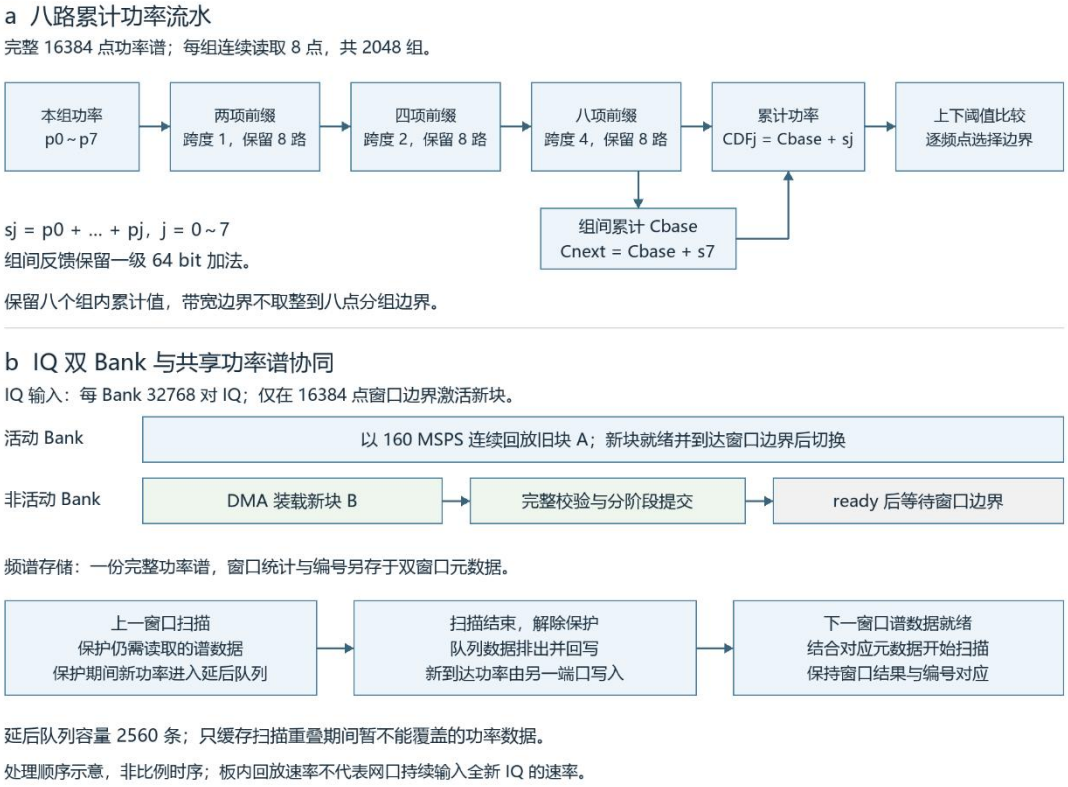
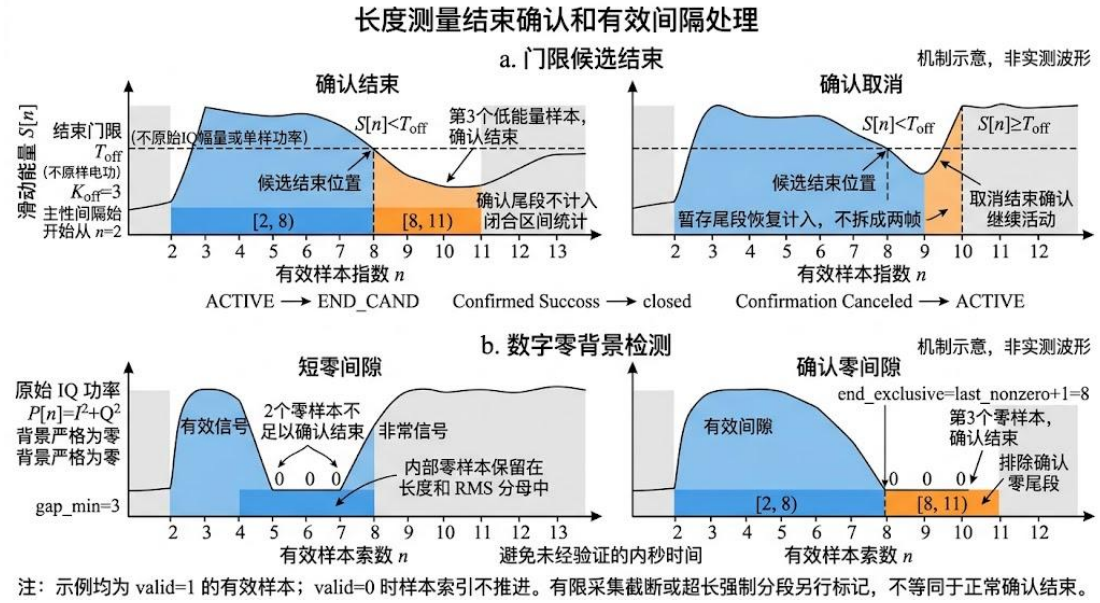


图 5-1 连续分析的并行处理与共享存储结构

亮点二：输入双 Bank 与共享功率谱协同，兼顾连续更新和存储复用。 IQ 双 Bank 允许非活动区接收新数据，活动区继续以 160 MSPS 回放；每个 Bank 保存 32768 对 IQ，新块仅在完整装载且当前 16384 点窗口结束后激活。频谱后处理采用单份完整功率谱，上一窗口扫描期间保护仍需读取的数据，暂不能覆盖的新功率进入延后写入队列，保护解除后排出回写。队列容量为 2560 条，双窗口元

数据另行保存统计量和编号。图 5-1b 展示了输入更新与频谱存储复用的配合关系。

亮点三：测长确认与有效区间分离。 门限检测进入候选结束状态后，暂存该位置及尾段能量、峰值；若连续低能量样本确认结束，按候选位置闭合区间，排除确认尾段；若信号恢复，则将暂存统计合并回活动区间，避免短暂跌落造成错误拆帧。如图 5-2 所示，数字零模式同样将结束确认与最后非零位置分开，保留不足确认长度的内部零间隙，并对截断、超长分段显式标记。区间采用左闭右开定义，valid=0 时不增加有效样本编号。图中确认长度取 3 仅用于说明机制，不代表实板测试参数。



注：示例均为 valid=1 的有效样本；valid=0 时样本索引不推进。有限采集截断或超长强制分段另行标记，不等同于正常确认结束。

图 5-2 测长结束确认与有效区间处理

亮点四：逐样本功率统计与共享数学运算分层。 高速通路只执行平方和、累计及峰值搜索，不为每个样本或频点配置开方单元。结果构建模块内部复用一套整数开方与除法单元，依次完成峰值幅度和 RMS；变长区间的平均功率同时保留整数商与余数，再生成定点开方输入，避免直接整除丢失小数信息。频谱模块的分位阈值另由自身除法器计算。时域统计使用未加窗 IQ，频域分支独立执行加窗与 FFT，使两类测量各自保持清晰的数值口径。

亮点五：系数重构与存储按需访问。 当前 Hann 预处理保存四分之一周期系数，通过索引、对称和补值逻辑重构完整系数序列，

减少重复存储；重构结果与参考实现进行等价性比较。回放 BRAM 仅在实际访问时使能，并将数据返回与 Bank 选择延迟对齐。

亮点六：提交与应答分阶段处理，保持事务一致性。 DMA 末尾提交分阶段处理，Bank 长度、编号、CRC 和 ready 状态统一更新；取消和错误事务不产生成功提交。ACK 的合法性判断先寄存，再更新消费者计数，避免同一事务重复执行。频率换算将常数乘法分解并安排在已有处理阶段内，DSP 总数保持 49。

6 总结与展望

6.1 成果与稳定性

本项目在 Zybo Z7-20 上实现了数字基带 IQ 信号分析系统，形成从上位机数据装载、PS DDR 缓冲、DMA 搬运到 PL 测量和结果回传的完整链路。输入采用 I、Q 各 16 bit 的有符号格式，PL 完成幅度、活动区间长度、峰频率和 99% 占用带宽计算，PS 与上位机负责控制、传输和显示。时域统计与频域处理分别使用明确的计算口径，输入 IQ 双 Bank、共享功率谱与延后写入队列配合八路扫描，使不同阶段能够重叠运行。

项目以 160 MSPS 进行板内回放，采用 16384 点 FFT。完整核心仿真归档完成了 1048576 个复数 FFT 点的位精确参考比较，产生 64 条频域记录。本轮实板完成 79 项数值检查、1000 次动态切换及 300 s 连续测试。1000 次切换中，发出、接受、FFT 输入和 FFT 输出样本计数均为 7967260672，输入拒绝、结果队列拒绝、DMA 错误和频域序号间隙均为零；实板最大分析与结果入环延迟分别为 219.775 μs、219.993750 μs。

当前版本另有 272 项宽带矩阵归档，主组满带子集的 99% 占用带宽为 158.154297～158.593750 MHz。2026 年 10 月 13 日经物理断电重上电后，系统独立完成 79 项数值复验和 300.016 s 连续测试，未在断电后重新下载程序。宽带矩阵、动态切换与冷启动分别记录，不合并为同一次验收。相对前期的 125 MSPS、8192 点版本，当前处理速率与 FFT 点数提高，但单窗分析延迟更长；相对相同点数的 150 MSPS 版本，最大分析延迟由 234.426667 μs 降至 219.775 μs。

当前设计能够在动态更新测试数据的同时保持板内连续处理，并通过计数、序号和错误状态检查数据完整性。其适用范围也有明确边界：输入来自数字测试向量回放，尚未接入外部采集前端，网口不持续提供 160 MSPS 的全新 IQ 数据；活动包络长度也不等于协议帧识别。冷启动后的数值复验已完成，但本轮 0 dB 含噪输入仍存在漏检，

低幅度及 Hann 边界突发等情况需分别分析。稳定性结论限于已测试的板卡、供电条件和运行时长。

6.2 后续优化路线

性能优化优先处理目前定位出的限制。当前完整设计在 160 MHz 下的建立时间裕量为 +0.007 ns，最差路径已转到频谱累计分位边界相关逻辑，由 cdf_bin_reg[5][3] 到 highq_reg[0] 的使能端，数据路径延迟为 5.827 ns。下一步可比较阈值比较、边界选择与使能逻辑的组织及布局，增加时序余量。DMA 分阶段提交及部分控制条件寄存化已完成，后续调整仍须同步检查窗口编号、边界结果和事务完整性，在现有频率下取得更稳定裕量后，再评估进一步提频。

数据更新效率需要在已有整体测量基础上进一步分解。本轮 1000 次切换和 300 s 连续测试的有效平均上传速率分别为 21.880890 Mbit/s、33.159468 Mbit/s，计时包含 UDP 装载、DMA 提交和等待激活。后续将上位机发送、DDR 收齐、缓存维护、DMA 搬运、Bank 激活和结果回传分别计时，区分测试脚本等待与实际传输开销，再判断发送窗口、重传策略或异步搬运是否值得调整。优化目标是缩短有效更新周期，同时保持板内处理和数据检查不回退。

测量能力的扩展则按资源和实际误差推进。当前频率间隔为 9.765625 kHz，非整数单音可评估谱峰插值，含噪突发可针对漏检和边界偏差研究噪声底估计与门限调整，需要协议帧识别时再加入前导相关。当前 BRAM 占用已达 91.43%，增加 FFT 点数、谱平均、更长缓存或并行通路前，应先核算共享存储、延后队列容量和服务周期。